ABSTRACT

Diabetic Retinopathy is one of the leading causes of preventable blindness among diabetic patients worldwide. Early and accurate detection of the disease is critical for timely medical intervention and vision preservation. Traditional screening methods rely on manual examination of retinal fundus images by trained ophthalmologists, a process that is time-consuming, costly, and difficult to scale, particularly in regions with limited access to specialists.

This project presents **DRetina Check**, a web-based deep learning application designed to automate the detection and classification of Diabetic Retinopathy from retinal fundus images. The system employs **Transfer Learning** using the **ResNet50** Convolutional Neural Network architecture pre-trained on the ImageNet dataset. The model is fine-tuned with custom dense classification layers to categorise retinal images into five internationally recognised severity classes: No Apparent Diabetic Retinopathy, Mild Non-Proliferative Diabetic Retinopathy, Moderate Non-Proliferative Diabetic Retinopathy, Severe Non-Proliferative Diabetic Retinopathy, and Proliferative Diabetic Retinopathy.

The application is built using **Python** and **Flask** for the backend, with **TensorFlow** and **Keras** powering the deep learning inference engine. The frontend is developed with **HTML5**, **CSS3**, **JavaScript**, and **Bootstrap** to provide an accessible and responsive user interface. Image preprocessing is handled using **Pillow** and **NumPy**, which resize and normalise uploaded images before feeding them into the model.

The system accepts retinal fundus images through the web interface, preprocesses them to the required 224 × 224 pixel format, performs model inference, and returns the predicted severity class along with a general clinical recommendation. Testing confirms that the end-to-end workflow, from image upload to result display, functions reliably and efficiently.

DRetina Check is intended as a preliminary screening support tool for healthcare professionals, rural screening centres, and academic research. It reduces diagnostic workload, improves screening accessibility, and demonstrates the effective application of artificial intelligence in medical image analysis.

Keywords: Diabetic Retinopathy, Deep Learning, Transfer Learning, ResNet50, Fundus Image Analysis, Medical Image Classification, Flask, TensorFlow.

TABLE OF CONTENTS

LIST OF FIGURES

Chapter 1

INTRODUCTION

1.1 Overview of the Project

Diabetic Retinopathy is a progressive eye disease caused by prolonged diabetes mellitus. It damages the blood vessels of the retina and can cause permanent vision loss if not detected at an early stage. According to the World Health Organization, Diabetic Retinopathy is one of the leading causes of preventable blindness globally. Since the disease shows minimal or no symptoms in its early stages, regular retinal screening is essential for all diabetic patients. Retinal fundus images are the standard diagnostic medium used by ophthalmologists to examine the retina. These images reveal visible disease indicators such as microaneurysms, haemorrhages, hard exudates, soft exudates, and abnormal blood vessel growth. Manual interpretation of these images requires highly trained specialists and considerable time,
which limits the scalability of traditional screening programmes.

DRetina Check is a web-based deep learning application developed to automate the classification of Diabetic Retinopathy from retinal fundus images. The system is designed to assist healthcare professionals and screening centres by providing fast and reliable prelimi- nary classification results. A user uploads a retinal image through a simple web interface. The system preprocesses the image, passes it through a trained deep learning model, and returns the predicted severity level along with a general recommendation.

The project uses **Transfer Learning** with the **ResNet50** Convolutional Neural Network architecture. ResNet50 is pre-trained on the large-scale ImageNet dataset and adapted for five-class Diabetic Retinopathy classification using custom dense layers. The backend is built with **Flask** and the deep learning inference is powered by **TensorFlow** and **Keras**. The frontend is developed using **HTML5**, **CSS3**, **JavaScript**, and **Bootstrap**. Image preprocessing tasks such as resizing, array conversion, and normalisation are handled using **Pillow** and **NumPy**.

The purpose of DRetina Check is not to replace ophthalmologists but to function as a supportive screening tool. It reduces manual workload, improves accessibility in resource- limited settings, and demonstrates the potential of artificial intelligence in healthcare.

1.2 Objective

The primary objective of this project is to develop a web-based application that automatically classifies Diabetic Retinopathy from retinal fundus images using deep learning and transfer learning techniques.
The specific objectives are listed below.

- To design a responsive and user-friendly web interface for retinal image upload and result display.
- To implement an image preprocessing pipeline that resizes and normalises uploaded fundus images to the format required by the model.
- To apply Transfer Learning using the ResNet50 architecture pre-trained on ImageNet for feature extraction from retinal images.
- To classify retinal fundus images into five internationally recognised Diabetic Retinopathy severity levels.
- To display the predicted severity class and a corresponding general clinical recommendation to the user.
- To reduce dependency on manual preliminary screening and improve acces- sibility to AI-assisted retinal analysis.
- To build a modular system architecture that supports future enhancements such as cloud deployment, Grad-CAM visualisation, and patient record man- agement.

1.3 Problem Statement

Diabetes mellitus is a chronic metabolic disorder affecting millions of people globally. One of its most serious complications is Diabetic Retinopathy, which progressively damages the retinal blood vessels and can lead to irreversible blindness if not diagnosed and treated in time. The disease passes through multiple severity stages, from mild non-proliferative changes to advanced proliferative retinopathy, and the transition between stages can be rapid without visible symptoms.

The conventional approach to Diabetic Retinopathy diagnosis relies on manual examination of retinal fundus photographs by qualified ophthalmologists. While this method is clinically reliable, it suffers from several critical limitations. The process is time-consuming and requires the continuous availability of trained specialists. In large-scale national or re-gional screening programmes, the volume of images to be reviewed is extremely high, creating a diagnostic bottleneck that delays timely intervention. In rural and semi-urban areas, including large parts of India, access to ophthalmologists is severely limited, making regular screening practically impossible for a significant portion of the diabetic population. Existing computer-aided diagnostic systems, where available, often require expensive specialised hardware, proprietary software licences, or deep integration with hospital in-formation systems, making them inaccessible for smaller clinics and community health centres. Many systems also perform only binary classification, indicating whether retinopa-thy is present or absent, without providing information about the severity level, which is critical for treatment planning.

There is therefore a clear and pressing need for an automated, accessible, accurate, and lightweight system that can perform multi-class Diabetic Retinopathy classification from standard retinal fundus images and provide preliminary results to support clinical decision-making.

DRetina Check addresses this need by providing a web-based deep learning solution that is platform-independent, requires no specialised hardware, and delivers five-class severity classification with clinical recommendations through a simple browser interface.

Chapter 2

LITERATURE REVIEW

2.1. Deep Learning for Detection and Classification of Diabetic Retinopathy in Reti-nal Fundus Images

This study investigates the use of deep convolutional neural networks for the automated detection and grading of Diabetic Retinopathy from retinal fundus photographs. The research demonstrates that CNN models can learn disease-relevant features, including microaneurysms, intraretinal haemorrhages, and neovascularisation, directly from labelled image data without manual feature design.

The study establishes that deep learning substantially outperforms traditional image processing pipelines in both sensitivity and specificity for Diabetic Retinopathy detection. It also highlights that the quality and size of the training dataset have a direct impact on model performance. Class imbalance, where images representing advanced disease stages are far fewer than normal images, is identified as a key challenge that requires careful handling through data augmentation or class weighting strategies.

The findings of this work directly inform the design of DRetina Check. The use of ResNet50 with transfer learning addresses the limited dataset challenge, and the five-class severity classification structure adopted in this project aligns with the grading framework validated in this research.

2.2. Automated Diabetic Retinopathy Screening Using Convolutional Neural Networks

This research presents a CNN-based automated screening system trained to distinguish between normal and Diabetic Retinopathy-affected retinal images. The study evaluates multiple CNN architectures and preprocessing configurations and demonstrates that normalisation and consistent image resizing are critical preprocessing steps that significantly affect model accuracy.

The paper identifies a major limitation of existing automated systems, namely that most perform only binary classification and cannot distinguish between severity levels. This limits their clinical utility because treatment decisions for Diabetic Retinopathy depend heavily on disease stage. A patient with mild non-proliferative retinopathy requires monitoring, whereas a patient with proliferative retinopathy requires urgent laser treatment or surgical intervention.

DRetina Check directly addresses this limitation by implementing five-class severity classification corresponding to the standard international grading scale, thereby providing clinically meaningful output rather than a simple normal or abnormal label.

2.3. Transfer Learning Approaches for Medical Image Classification

This work provides a comprehensive review of transfer learning strategies applied to medical image classification tasks including chest X-ray analysis, skin lesion classification, histopathology grading, and retinal disease detection. The study compares several pre-trained architectures including VGG16, VGG19, InceptionV3, DenseNet121, and ResNet50 across multiple medical imaging benchmarks.

The research concludes that ResNet50 offers a strong balance between classification accuracy and computational efficiency, making it suitable for deployment in resource-constrained environments. The residual connections in ResNet50 mitigate the vanishing gradient problem and allow effective training of deeper layers, which is important for learning fine-grained features in medical images.

The study also recommends fine-tuning the final fully connected layers while freezing the convolutional base during initial training, a strategy adopted in DRetina Check to leverage pre-trained ImageNet features for retinal image feature extraction.

2.4. ResNet-Based Deep Learning Model for Retinal Disease Detection

This study focuses specifically on the application of Residual Network architectures for retinal disease classification including Diabetic Retinopathy, glaucoma, and age-related macular degeneration. The research demonstrates that the skip connections in ResNet architectures allow gradients to flow more effectively through deep networks, enabling the model to capture both low-level edge and texture features and high-level semantic disease patterns in retinal images.

The paper reports that ResNet50 achieves competitive accuracy on retinal disease classification benchmarks while requiring significantly fewer parameters than deeper architectures such as ResNet101 or ResNet152. This makes ResNet50 a practical choice for systems where inference speed and memory efficiency are important considerations.

In DRetina Check, ResNet50 serves as the feature extraction backbone. The pre-trained convolutional layers extract rich visual representations from retinal fundus images, and custom dense layers appended to the network perform the five-class Diabetic Retinopathy classification.

2.5. Artificial Intelligence in Ophthalmology for Early Diagnosis of Diabetic Eye Diseases

This study reviews the growing role of artificial intelligence in ophthalmology and examines AI-based systems for the early detection of Diabetic Retinopathy, glaucoma, diabetic macular oedema, and age-related macular degeneration. The research analyses clinical validation studies of AI diagnostic tools and discusses their integration into real-world healthcare workflows.

A central finding of this study is that AI-based screening systems are most effective when positioned as decision-support tools that complement clinical expertise rather than replace it. The study recommends that automated systems provide not only a classification output but also an indication of confidence and a recommendation for further clinical evaluation when the predicted severity is moderate or above.

DRetina Check adopts this philosophy by providing severity-specific general recommendations alongside the predicted class label, thereby directing users toward appropriate medical consultation rather than presenting the output as a definitive diagnosis.

Chapter 3

EXISTING SYSTEMS AND THEIR DRAWBACKS

This chapter examines the current approaches used for Diabetic Retinopathy detection and screening. Understanding the limitations of existing systems provides the justification for the proposed DRetina Check solution.

3.1 Manual Ophthalmological Examination

The most widely used method for Diabetic Retinopathy diagnosis involves direct examination of the retina by qualified ophthalmologists using specialised instruments such as slit-lamp biomicroscopy or indirect ophthalmoscopy. Retinal fundus photographs captured using a fundus camera are also commonly reviewed manually by trained graders.
In this approach, the clinician examines the retinal image for characteristic disease signs including microaneurysms, dot and blot haemorrhages, hard exudates, cotton wool spots, venous beading, and neovascularisation. Based on the findings, the clinician assigns a severity grade and recommends appropriate treatment or monitoring.

Drawbacks

- **Specialist dependency:** Manual grading requires the availability of trained ophthalmologists or certified retinal graders, who are not uniformly distributed across urban and rural regions.
- **Time-consuming:** Manual review of large volumes of fundus images in population-level screening programmes creates significant diagnostic delays.
- **Scalability limitations:** The method cannot be efficiently scaled to meet the growing global burden of diabetes and the corresponding demand for retinal screening.
- **Inter-grader variability:** Differences in training, experience, and clinical judgement between graders can lead to inconsistent severity classifications for the same image.

- **Geographic inaccessibility:** In rural and semi-urban areas of India and other developing countries, patients must travel long distances to access retinal screening facilities, leading to poor screening uptake and delayed diagnosis.

3.2 Traditional Computer-Aided Detection Systems

Earlier computer-aided detection systems for Diabetic Retinopathy used classical image processing algorithms. These systems applied techniques such as green channel extraction, contrast-limited adaptive histogram equalisation, morphological operations, and thresholding to segment retinal features and detect lesion regions.
Representative systems in this category include the Messidor project and early implementations of lesion-specific detectors for microaneurysms and haemorrhages.

Drawbacks

- **Manual feature engineering:** These systems require expert-designed algorithms for each type of retinal lesion, making development and maintenance complex and time-intensive.
- **Poor generalisation:** Performance degrades significantly when images are captured using different fundus cameras, lighting conditions, or patient demographics not represented in the development dataset.
- **Binary output only:** Most traditional systems classify images as either normal or abnormal and do not provide multi-class severity grading, limiting their clinical relevance.
- **Sensitivity to image quality:** These methods are highly sensitive to noise, poor contrast, motion blur, and uneven illumination, which are common in real-world fundus images.
- **High false positive rates:** Morphological feature detectors frequently misclassify normal retinal structures as lesions, requiring additional manual review to filter false positives.

3.3 Commercial AI-Based Screening Platforms

Several commercial AI-based Diabetic Retinopathy screening solutions are currently available, including IDx-DR, EyeArt, Retmarker, and Google's diabetic retinopathy detection system. These platforms have been clinically validated and are approved for use in specific healthcare contexts.

Drawbacks

- **High cost:** Commercial platforms require expensive software licences or subscription fees, making them inaccessible for smaller clinics, community health centres, and low-income healthcare settings.
- **Hardware dependency:** Several commercial systems are tightly integrated with specific fundus camera models, restricting their use to facilities with compatible equipment.
- **Limited customisability:** Commercial solutions are closed systems and cannot be easily adapted for research purposes, extended to detect additional retinal diseases, or integrated with local hospital information systems.
- **Internet connectivity requirements:** Cloud-based commercial platforms require reliable high-speed internet access, which is not consistently available in rural screening settings.
- **Regulatory restrictions:** Many commercial systems are approved only for specific clinical use cases and geographic regions, limiting their broader applicability.

3.4 Comparative Summary of Existing Systems

System Type	Approach	Key Drawbacks
Manual Examination	Ophthalmologist review of fundus images	Specialist dependency, slow, poor scalability, geographic inaccessibility
Traditional CAD Systems	Classical image processing and morphological feature detection	Manual feature engineering, binary output, poor generalisation, high false positives
Commercial AI Platforms	Deep learning with proprietary models and cloud infrastructure	High cost, hardware dependency, limited customisability, connectivity requirements
DRetina Check	Transfer learning with ResNet50, open-source, browser-based	Addresses all above drawbacks; lightweight, accessible, multi-class, extendable

Table 3.1: Comparison of Existing Systems

Chapter 4

PROPOSED SYSTEM AND ITS ADVANTAGES

4.1 Proposed System

DRetina Check is a web-based deep learning application that automates the detection and classification of Diabetic Retinopathy from retinal fundus images. The system is designed to be lightweight, platform-independent, and accessible through any standard web browser without requiring specialised hardware or proprietary software.

The proposed system follows a modular architecture consisting of five components: a responsive web-based user interface, a Flask backend server, an image preprocessing pipeline, a ResNet50-based deep learning inference engine, and a recommendation generation module.

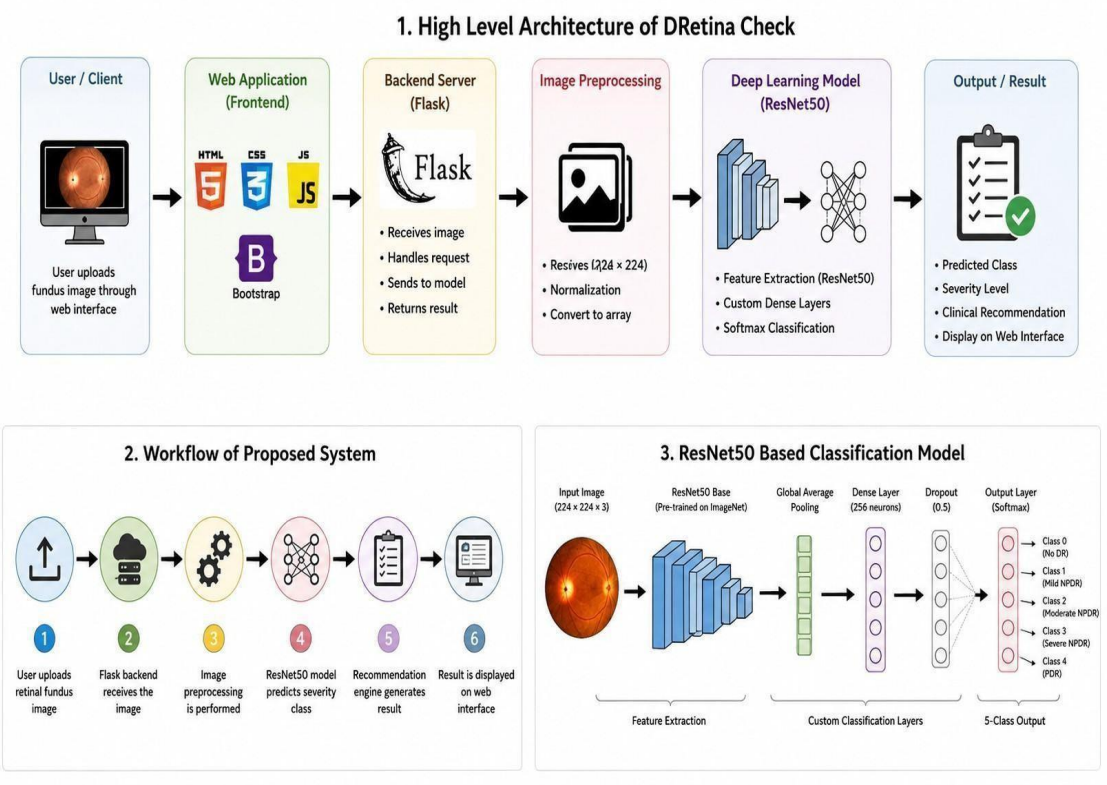


Figure 4.1: High Level Architecture of DRetina Check

4.2 System Workflow

The operational workflow of DRetina Check is described below.

1. User uploads retinal fundus image.
2. Flask backend receives image.
3. Image preprocessing is performed.
4. ResNet50 model predicts severity class.
5. Recommendation engine generates result.
6. Result is displayed on web interface.

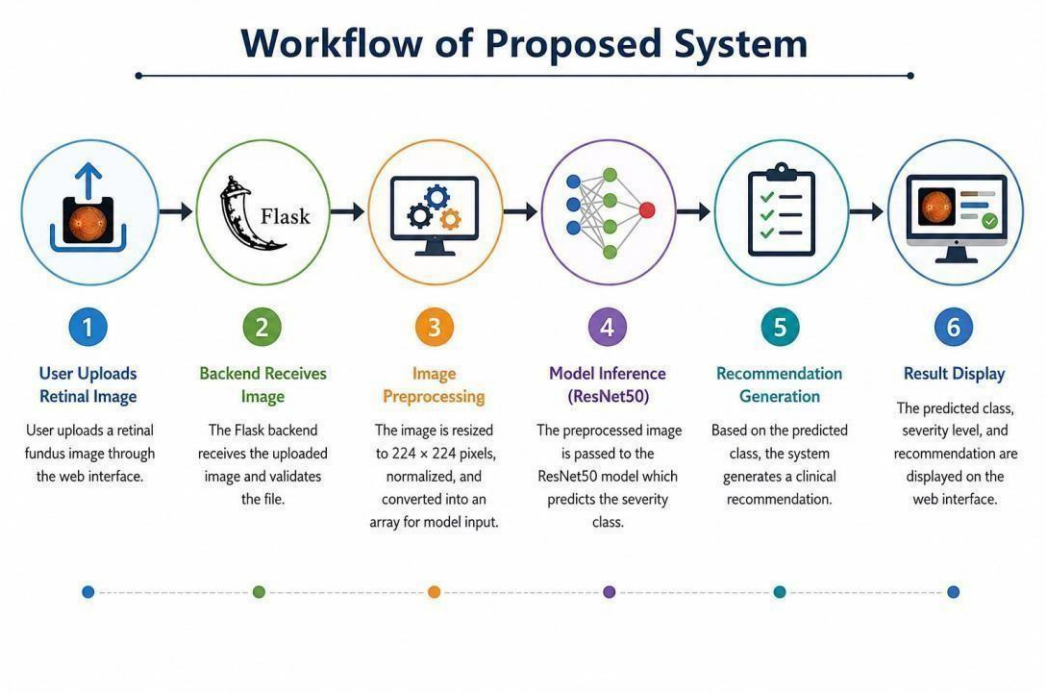


Figure 4.2: Workflow of Proposed System

4.3 Deep Learning Model

The classification model is built using Transfer Learning with the ResNet50 Convolutional Neural Network architecture.
ResNet50 is pre-trained on ImageNet and acts as a feature extractor. The final layers are replaced with custom dense layers for Diabetic Retinopathy classification.
The architecture consists of:

- ResNet50 Base Model

- Global Average Pooling Layer
- Dense Layer (256 neurons)
- Dropout Layer (0.5)
- Softmax Output Layer

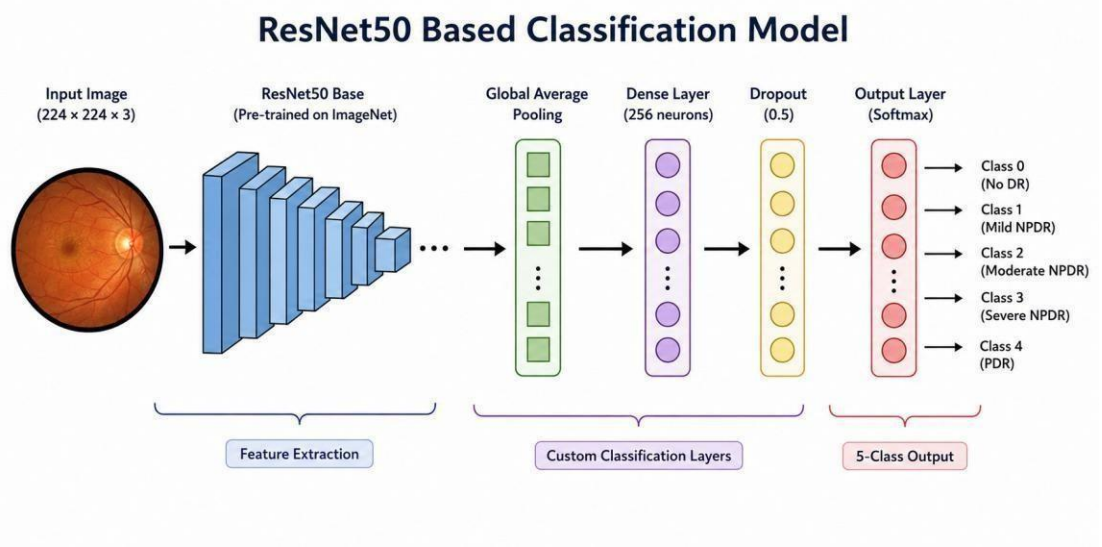


Figure 4.3: ResNet50 Based Classification Model

4.4 Classification Categories

The system classifies retinal fundus images into five severity levels.

Class	**Severity Level**	**Clinical Significance**
Class 0	No Apparent Diabetic Retinopathy	No visible retinal lesions. Annual screening recommended.
Class 1	Mild NPDR	Microaneurysms detected. Monitoring advised.
Class 2	Moderate NPDR	Ophthalmologist consultation recommended.
Class 3	Severe NPDR	Urgent specialist referral required.
Class 4	Proliferative Diabetic Retinopathy	Immediate medical intervention required.

Table 4.1: Diabetic Retinopathy Classification Categories

4.5 Recommendation Module

Based on the predicted class, the system generates recommendations.

Class 0	Maintain regular annual screening.
Class 1	Consult ophthalmologist and maintain glycaemic control.
Class 2	Seek detailed ophthalmological evaluation.
Class 3	Urgent referral to retinal specialist recommended.
Class 4	Immediate medical consultation required.

Table 4.2: Severity-Based Clinical Recommendations

4.6 Advantages of the Proposed System

4.6.1 Automated Multi-Class Classification

Unlike traditional systems, DRetina Check performs five-class classification and provides clinically meaningful output.

4.6.2 Transfer Learning for Limited Data

ResNet50 enables accurate feature extraction even with limited medical datasets.

4.6.3 Lightweight and Platform Independent

The application runs on any device supporting a web browser.

4.6.4 Fast Inference

Prediction results are generated within seconds.

4.6.5 Modular and Extensible Architecture

Future enhancements can be integrated without major redesign.

Chapter 5

SYSTEM REQUIREMENTS AND SPECIFICATIONS

System requirements define the hardware, software, functional, and non-functional conditions necessary for developing and running DRetina Check. These requirements ensure that the application performs reliably and efficiently across the intended deployment environments.

5.1 Software Requirements

The software requirements include all technologies, frameworks, libraries, and tools used in the development and execution of the application.

Software Component	Description and Justification
Operating System	Windows 10/11 or Ubuntu Linux 20.04 or later. The application is platform-independent and runs on both operating systems.
Programming Language	Python 3.8 or later. Chosen for its extensive support for machine learning, deep learning, image processing, and web development.
Backend Framework	Flask 2.x and Flask-CORS. Flask provides a lightweight WSGI web framework suitable for integrating Python-based deep learning models with a web interface.
Deep Learning Framework	TensorFlow 2.x and Keras. Used for loading the trained ResNet50 model and performing image classification inference.
Frontend Technologies	HTML5, CSS3, JavaScript, and Bootstrap 5. Used for designing the responsive user interface for image upload and result display.
Image Processing Libraries	Pillow (PIL) for image opening, resizing, and format conversion. NumPy for converting images into numerical arrays and normalising pixel values.

Model Architecture	ResNet50 with pre-trained ImageNet weights via Keras Applications. Custom dense layers added for five-class classification.
Development Tools	Visual Studio Code for application development. Jupyter Notebook for model training, experimentation, and evaluation.
Web Browser	Google Chrome, Mozilla Firefox, Microsoft Edge, or any modern standards-compliant browser.

Table 5.1: Software Requirements

5.2 Hardware Requirements

The hardware requirements define the minimum and recommended system configuration for running the DRetina Check application.

Hardware Component	**Specification**
Processor	Intel Core i3 or i5 (8th generation or later) or equivalent AMD processor. A multi-core processor is recommended for efficient model inference.
Memory (RAM)	Minimum 4 GB RAM. 8 GB or more recommended for smooth application performance during model loading and prediction.
Storage	Minimum 2 GB of free disk space for the application, trained model file, and dependencies. SSD storage recommended for faster model loading.
Display	Standard monitor with a minimum resolution of 1280 × 720 pixels with browser support.
GPU	Optional for inference. An NVIDIA GPU with CUDA support is recommended for faster model training during the development phase.
Network	Local network or localhost connection sufficient for running the Flask development server. Internet connection required only for initial dependency installation.

Table 5.2: Hardware Requirements

5.3 Functional Requirements

Functional requirements describe the specific behaviours and capabilities that the system must provide.

FR	Requirement Description
FR1	The system shall provide a web-based user interface accessible through a standard browser.
FR2	The system shall allow the user to upload a retinal fundus image in JPEG or PNG format.
FR3	The system shall validate the uploaded file and reject unsupported file formats with an appropriate error message.
FR4	The system shall resize the uploaded image to 224 × 224 pixels as required by the ResNet50 input specification.
FR5	The system shall convert the resized image into a numerical array and normalise pixel values to the range [0, 1].
FR6	The system shall load the trained ResNet50-based classification model at application startup.
FR7	The system shall pass the preprocessed image to the model and generate probability scores for all five severity classes.
FR8	The system shall identify and return the class with the highest probability score as the predicted severity level.
FR9	The system shall display the predicted Diabetic Retinopathy class label clearly on the result page.
FR10	The system shall display a general clinical recommendation corresponding to the predicted severity class.

Table 5.3: Functional Requirements

5.4 Non-Functional Requirements

Non-functional requirements define the quality attributes and operational constraints of the system.

5.4.1 Accuracy

The system shall provide reliable classification results based on the trained deep learning model. Model accuracy is dependent on the quality and diversity of the training dataset, the preprocessing pipeline, and the ResNet50 architecture. The system is intended for preliminary screening support and results should be verified by a qualified medical professional.

5.4.2 Performance

The system shall generate prediction results within five seconds of image submission under normal operating conditions on the minimum specified hardware configuration. Model loading at application startup shall not affect per-request inference time.

5.4.3 Usability

The user interface shall be simple, intuitive, and accessible to users without technical knowledge of deep learning or medical imaging. All interface labels, result displays, and recommendations shall be written in clear, non-technical language.

5.4.4 Reliability

The system shall handle valid image inputs correctly and produce consistent classification results for identical inputs. It shall manage invalid uploads, unsupported file formats, and missing file submissions gracefully without application crashes.

5.4.5 Maintainability

The system shall follow a modular architecture that clearly separates the frontend, backend, preprocessing, and inference components. Each module shall be independently updatable without requiring changes to other modules.

5.4.6 Scalability

The system architecture shall support future enhancements including cloud deployment on platforms such as AWS or Google Cloud, integration of patient record management, Grad-CAM explainability visualisation, mobile application support, and extension to classify additional retinal diseases beyond Diabetic Retinopathy.

5.4.7 Security

Uploaded image files shall be processed in memory and shall not be permanently stored on the server in the current version. Future versions incorporating patient record management shall implement role-based access control and encrypted data storage.

Chapter 6

SYSTEM DESIGN ANALYSIS

System design analysis describes the structural and behavioural architecture of DRetina Check. It explains how the individual components of the system are organised and how they interact with each other to accomplish the end-to-end Diabetic Retinopathy classification workflow. The design follows a layered modular approach that separates concerns across the user interface, application server, preprocessing pipeline, and deep learning inference engine.

6.1 High Level Architecture

The high-level architecture of DRetina Check consists of four layers. Each layer performs a distinct role and communicates with adjacent layers through well-defined interfaces.

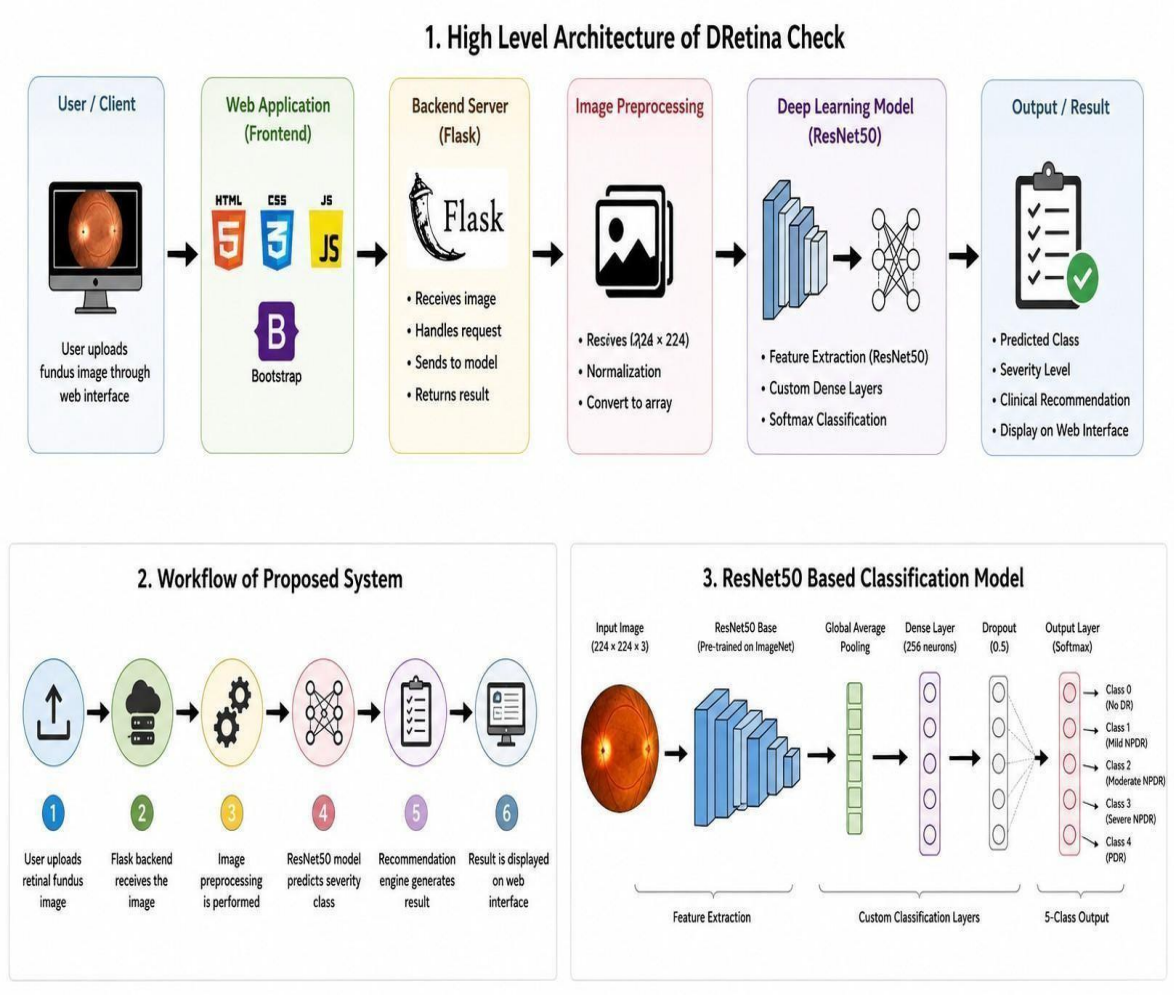


Figure 6.1: High Level Architecture of DRetina Check

The **User Layer** serves as the entry point of the DRetina Check system. It provides a simple and user-friendly interface through which users can interact with the application. Using a web browser, the user can access the application, upload a retinal fundus image, and submit it for analysis. The interface is designed to be intuitive and accessible, allowing even non-technical users to perform the screening process easily. Once the image is uploaded, the user waits for the system to process the image and display the prediction results along with the corresponding recommendation.

The **Application Layer** is managed by the Flask backend server, which acts as the central controller of the system. It receives HTTP requests generated from the frontend, validates the uploaded image file, and ensures that the input meets the required format and quality standards. After successful validation, the backend coordinates the entire workflow by invoking the image preprocessing module and the deep learning inference engine. The Flask server also manages communication between different system components and ensures smooth data transfer from image upload to result generation.

The **Processing Layer** contains the image preprocessing module and the deep learning classification model. The preprocessing module prepares the uploaded retinal image by performing operations such as resizing it to 224 × 224 pixels, converting it into a numerical array, and normalizing pixel values. These steps ensure that the image matches the input requirements of the trained model. The processed image is then passed to the ResNet50-based deep learning classifier, which extracts meaningful visual features from the retinal image and analyzes disease-related patterns. Using transfer learning and trained classification layers, the model generates probability scores for each of the five Diabetic Retinopathy severity classes and identifies the most probable category.

The **Output Layer** is responsible for interpreting and presenting the prediction results in a user-friendly format. The probability scores produced by the model are mapped to their corresponding Diabetic Retinopathy severity levels, ranging from No Apparent Diabetic Retinopathy to Proliferative Diabetic Retinopathy. Based on the predicted class, the system generates an appropriate recommendation to guide the user regarding further medical consultation. The final result, including the predicted severity level and recommendation, is transmitted back to the frontend interface and displayed clearly to the user. This layer ensures that complex model outputs are converted into meaningful information that can assist in preliminary screening and decision-making.

6.2 Low Level Design

The low-level design describes the internal step-by-step flow of the application from the moment the user submits an image to the moment the result is displayed.

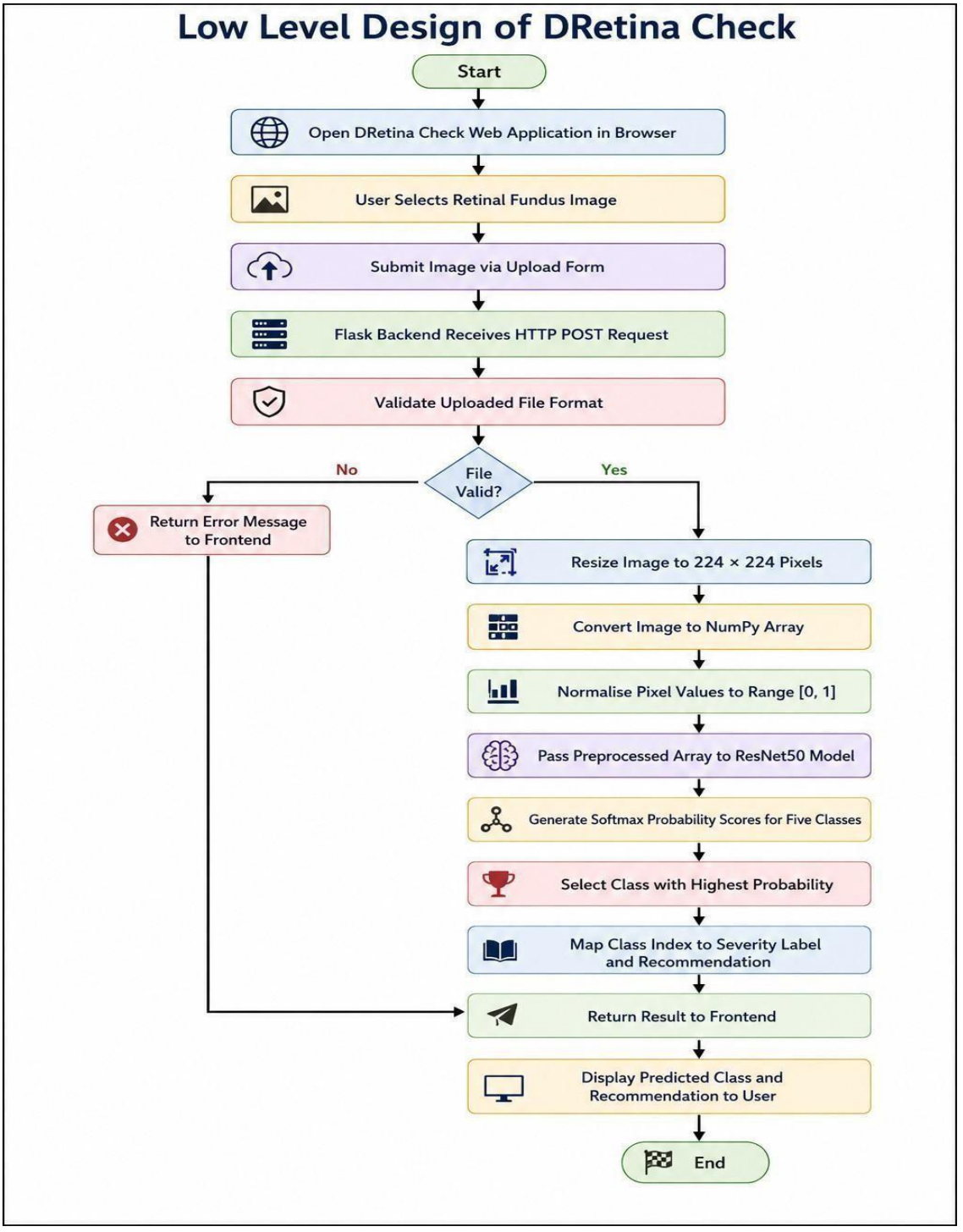


Figure 6.2: Low Level Design of DRetina Check

The workflow of DRetina Check begins when the user accesses the web application through a browser and uploads a retinal fundus image for analysis. The uploaded image is sent to the Flask backend server, which receives the request and performs initial validation checks. These checks ensure that a file has been selected, the uploaded file is in a supported image format, and the input can be processed safely by the system. If the uploaded file is invalid or corrupted, the system immediately returns an appropriate error message to the user and prevents further processing.

For valid retinal images, the preprocessing module prepares the image for deep learning inference. The image is first resized to the dimensions required by the trained ResNet50 model, ensuring consistency with the model's input specifications. It is then converted into a NumPy array so that it can be processed computationally. After conversion, pixel values are normalized to a standard range, improving model performance and ensuring that the input data matches the format used during training. These preprocessing operations help remove inconsistencies and prepare the image for accurate classification.

Once preprocessing is completed, the processed image is forwarded to the ResNet50-based deep learning classifier. The model extracts important visual features from the retinal image, such as patterns related to blood vessels, lesions, hemorrhages, microaneurysms, and other disease indicators. Using the learned features and trained classification layers, the model analyzes the image and generates probability scores for each of the five Diabetic Retinopathy severity classes. A Softmax output layer is used to calculate the probability distribution across all classes, enabling the model to determine the most likely severity level.

The predicted class is then selected based on the highest probability score and mapped to its corresponding Diabetic Retinopathy severity label. The system also generates a suitable recommendation based on the predicted severity level, providing guidance regarding further medical consultation or follow-up screening. Finally, the prediction result and recommendation are transmitted back to the frontend interface, where they are displayed clearly to the user. This complete workflow enables automated retinal image analysis and provides a fast, user-friendly solution for preliminary Diabetic Retinopathy screening.

6.3 Data Flow Diagram

The Data Flow Diagram illustrates the movement of data through the system from the user input to the final output displayed on the web interface.

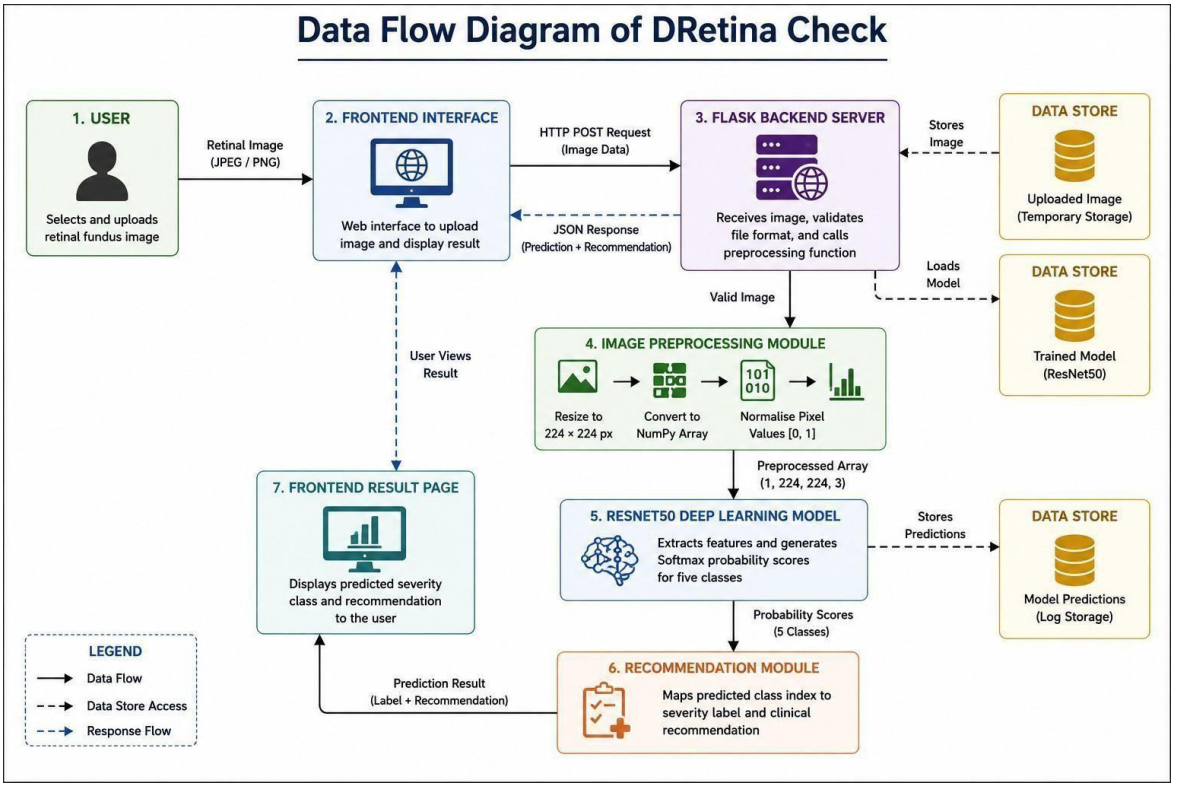


Figure 6.3: Data Flow Diagram of DRetina Check

The user first uploads a retinal fundus image through the frontend interface. The image is transmitted to the Flask backend server using an HTTP POST request. The backend validates the uploaded file and forwards it to the preprocessing module.

The preprocessing module resizes the image, converts it into a NumPy array, and normalises the pixel values. The processed image is then supplied to the ResNet50 deep learning model for feature extraction and classification.

The model produces probability scores for all five Diabetic Retinopathy classes. The recommendation module interprets the predicted class and maps it to the appropriate severity level and clinical recommendation. The final result is then sent back to the frontend and displayed to the user.

6.4 Sequence Diagram

The sequence diagram represents the chronological order of interactions between the system components during a single image upload and classification operation.

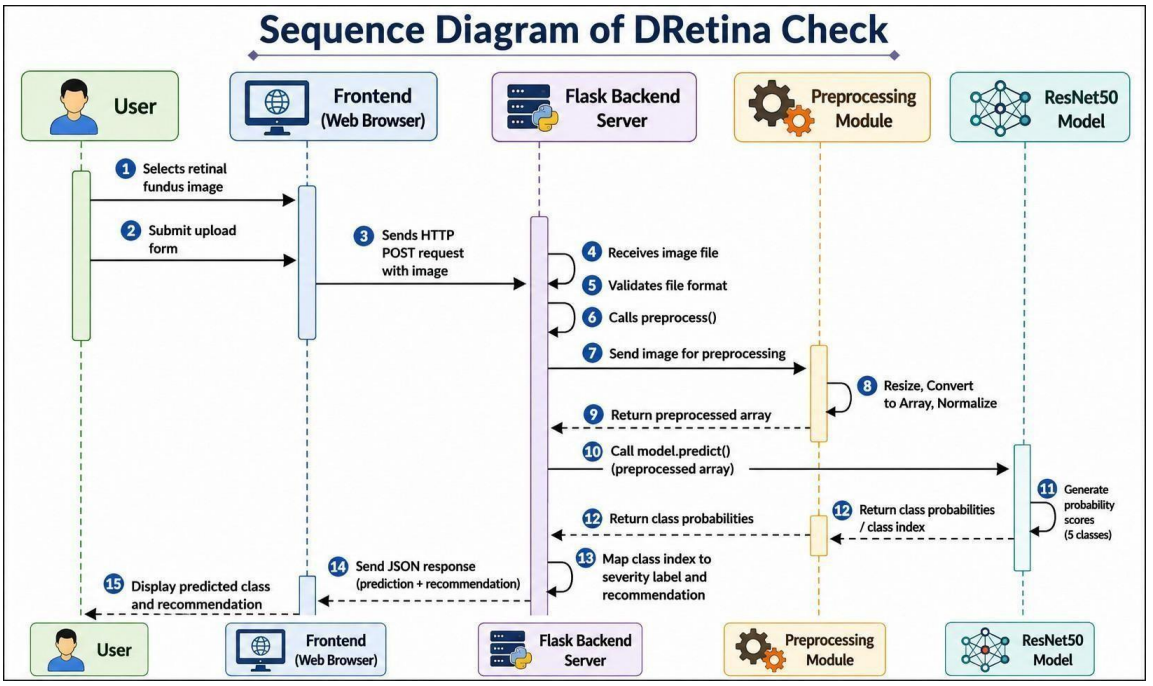


Figure 6.4: Sequence Diagram of DRetina Check

The process begins when the user selects a retinal fundus image and submits it through the upload form available on the web interface. The frontend sends an HTTP request containing the image to the Flask backend server.

The backend validates the uploaded image and initiates the preprocessing stage. The preprocessing module resizes the image to the required dimensions, converts it into a NumPy array, and performs pixel normalisation.

Once preprocessing is complete, the processed image is passed to the ResNet50 classification model. The model performs inference and generates probability scores corresponding to the five severity classes.

The backend receives the predicted class index, maps it to the corresponding severity label and recommendation, and returns the result to the frontend. Finally, the frontend renders the prediction and recommendation for the user.

6.5 Model Pipeline

The model pipeline describes the complete transformation applied to the input retinal fundus image from the point of upload to the generation of a severity class prediction.

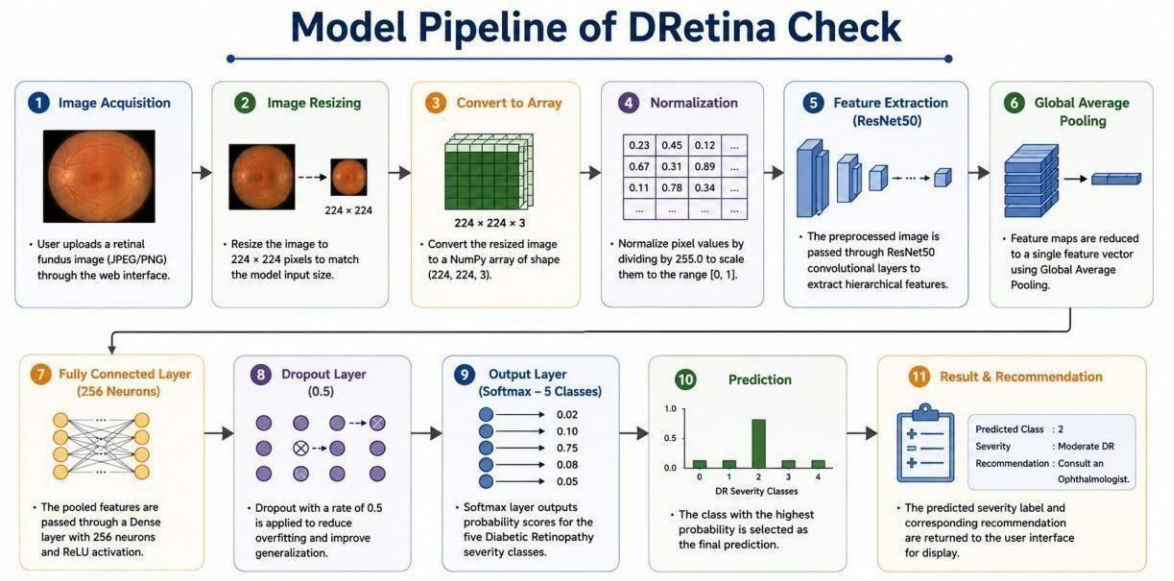


Figure 6.5: Model Pipeline of DRetina Check

The pipeline begins with the acquisition of a retinal fundus image uploaded by the user through the web interface. The uploaded image may be in JPEG or PNG format and can originate from different retinal imaging devices.

The first stage involves image resizing, where the original image is transformed to a fixed dimension of 224 × 224 pixels. This resizing step ensures compatibility with the ResNet50 architecture, which expects images of a fixed input size.

After resizing, the image is converted into a NumPy array representation. The pixel values are then normalised by dividing each value by 255.0, scaling the data into the range [0, 1]. Normalisation improves numerical stability and helps the neural network converge effectively.

The normalised image is passed through the convolutional layers of the ResNet50 model. These layers automatically extract hierarchical visual features including edges, textures, blood vessel structures, lesion patterns, and other retinal abnormalities relevant to Diabetic Retinopathy classification.

The extracted feature maps are processed through a Global Average Pooling layer, followed by a Dense layer containing 256 neurons with ReLU activation. A Dropout layer with a rate of 0.5 is applied to reduce overfitting and improve model generalisation.

Finally, a Softmax output layer containing five neurons generates probability scores corresponding to the five Diabetic Retinopathy severity classes. The class with the highest probability score is selected as the final prediction and returned to the user interface.

6.6 Module Architecture

The DRetina Check system is composed of multiple independent modules that work together to perform image classification and recommendation generation. Each module is responsible for a specific function within the overall system.

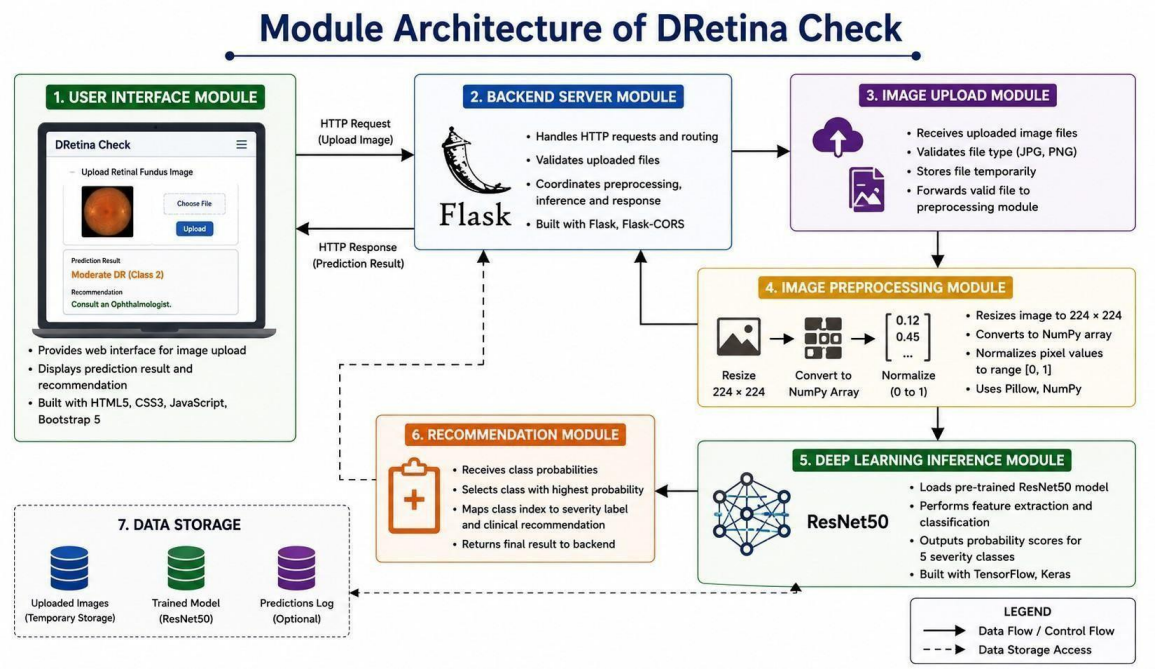


Figure 6.6: Module Architecture of DRetina Check

The modular architecture improves maintainability, scalability, and ease of future enhancement. Since individual components are loosely coupled, updates can be implemented without affecting unrelated parts of the application.

Module	Technology	Responsibility
User Interface Module	HTML5, CSS3, JavaScript, Bootstrap 5	Provides the image upload form and result display page. Ensures responsive layout across desktop and mobile devices.
Backend Server Module	Flask, Flask-CORS	Handles HTTP routing, request processing, file validation, and communication between preprocessing and inference modules.

Image Upload Module	Flask File Handling	Manages uploaded image files, validates supported formats, and forwards valid files for preprocessing.
Image Preprocessing Module	Pillow, NumPy	Resizes images, converts them into NumPy arrays, and normalises pixel values before model inference.
Deep Learning Inference Module	TensorFlow, Keras, ResNet50	Loads the trained model, performs classification inference, and generates probability scores for five severity classes.
Recommendation Module	Python Dictionary Mapping	Maps predicted class indices to severity labels and corresponding clinical recommendations for display.

Table 6.1: Module Architecture of DRetina Check

The User Interface Module serves as the interaction point between the user and the system. The Backend Server Module coordinates all system operations and ensures smooth communication between different components.

The Image Upload and Preprocessing Modules prepare uploaded images for analysis. The Deep Learning Inference Module performs feature extraction and classification using the ResNet50 architecture. Finally, the Recommendation Module converts classification results into meaningful recommendations that assist users in understanding the predicted severity level.

Chapter 7

Implementation

7.1 : Data Description

The DRetina Check system is trained using the **Diabetic Retinopathy Level Detection Dataset** obtained from Kaggle. The dataset consists of preprocessed retinal fundus images collected for the purpose of diabetic retinopathy severity classification.

The objective of the dataset is to enable automated detection and grading of Diabetic Retinopathy using deep learning techniques. Each retinal image is labelled according to the severity of retinal damage caused by diabetes.

The dataset contains images belonging to **five distinct classes**, representing different stages of Diabetic Retinopathy progression.

Class Label	Severity Level
Class 0	No Apparent Diabetic Retinopathy
Class 1	Mild Non-Proliferative Diabetic Retinopathy (Mild NPDR)
Class 2	Moderate Non-Proliferative Diabetic Retinopathy (Moderate NPDR)
Class 3	Severe Non-Proliferative Diabetic Retinopathy (Severe NPDR)
Class 4	Proliferative Diabetic Retinopathy (PDR)

The trained deep learning model learns disease-related retinal features such as microaneurysms, haemorrhages, exudates, vascular abnormalities, and neovascularization from the retinal fundus images. Based on these features, the model predicts the most probable severity stage of Diabetic Retinopathy.The dataset used in this project contains:

- Training Images: **3626**
- Validation Images: **726**
- Total Images Used: **4352**
- Number of Classes: **5**
- Image Input Size: **224 × 224 pixels**

The primary goal of the model is to classify an uploaded retinal fundus image into one of the five diabetic retinopathy severity categories and provide a corresponding recommendation for further clinical evaluation.

7.1.1 : Loading the dataset into the Google colab:

We are going to clone the Kaggle dataset on colab. Kindly refer to this - https://www.analyticsvidhya.com/blog/2021/06/how-to-load-kaggle-datasets-directly-into-google-colab/

- Clone kaggle in google colab

```
from mpl_toolkits.mplot3d import Axes3D
from sklearn.preprocessing import StandardScaler
import matplotlib.pyplot as plt # plotting
import numpy as np # linear algebra
import os # accessing directory structure
import pandas as pd # data processing, CSV file I/O (e.g. pd.read_csv)
```

```
[ ] !pip install -q kaggle
```

```
[ ] !mkdir ~/.kaggle
```

```
[ ] !cp kaggle.json ~/.kaggle
```

```
[ ] !chmod 600 /root/.kaggle/kaggle.json
```

- Download the dataset

```
!kaggle datasets download -d arbethi/diabetic-retinopathy-level-detection
```

```
Downloading diabetic-retinopathy-level-detection.zip to /content
100% 9.65G/9.66G [01:22<00:00, 83.0MB/s]
100% 9.66G/9.66G [01:22<00:00, 125MB/s]
```

- Unzip the dataset

```
!unzip /content/diabetic-retinopathy-level-detection.zip

Archive:  /content/diabetic-retinopathy-level-detection.zip
  inflating: inception-diabetic.h5
  inflating: preprocessed dataset/preprocessed dataset/testing/0/cfb17a7cc8d4.png
  inflating: preprocessed dataset/preprocessed dataset/testing/0/cfdbaef73a8b.png
  inflating: preprocessed dataset/preprocessed dataset/testing/0/cfed7c1172ec.png
  inflating: preprocessed dataset/preprocessed dataset/testing/0/cff262ed8f4c.png
  inflating: preprocessed dataset/preprocessed dataset/testing/0/cffc50047828.png
  inflating: preprocessed dataset/preprocessed dataset/testing/0/d02b79fc3200.png
  inflating: preprocessed dataset/preprocessed dataset/testing/0/d0926ed2c8e5.png
  inflating: preprocessed dataset/preprocessed dataset/testing/0/d160ebef4117.png
  inflating: preprocessed dataset/preprocessed dataset/testing/0/d16e39b9d6f0.png
  inflating: preprocessed dataset/preprocessed dataset/testing/0/d16e59a2b33a.png
  inflating: preprocessed dataset/preprocessed dataset/testing/0/d18f6431ebce.png
  inflating: preprocessed dataset/preprocessed dataset/testing/0/d1a60c3b9fe5.png
  inflating: preprocessed dataset/preprocessed dataset/testing/0/d1afdb8cf70d.png
  inflating: preprocessed dataset/preprocessed dataset/testing/0/d1b279cc02ae.png
  inflating: preprocessed dataset/preprocessed dataset/testing/0/d1ca85af57c9.png
  inflating: preprocessed dataset/preprocessed dataset/testing/0/d1cf31577a59.png
  inflating: preprocessed dataset/preprocessed dataset/testing/0/d1f7ea924a01.png
  inflating: preprocessed dataset/preprocessed dataset/testing/0/d1fa0f744620.png
  inflating: preprocessed dataset/preprocessed dataset/testing/0/d26bb2ed6e71.png
  inflating: preprocessed dataset/preprocessed dataset/testing/0/d26bc6e1230d.png
  inflating: preprocessed dataset/preprocessed dataset/testing/0/d27ac9e54901.png
  inflating: preprocessed dataset/preprocessed dataset/testing/0/d29b37d110f3.png
  inflating: preprocessed dataset/preprocessed dataset/testing/0/d2afca74cbc3.png
  inflating: preprocessed dataset/preprocessed dataset/testing/0/d2c5fb82fe5f.png
  inflating: preprocessed dataset/preprocessed dataset/testing/0/d2cd47ed2c1d.png
  inflating: preprocessed dataset/preprocessed dataset/testing/0/d2d523e9f669.png
  inflating: preprocessed dataset/preprocessed dataset/testing/0/d2dc86021c67.png
```

7.1.2 : Create Training & Testing Path:

To build a TL model we have to split training and testing data into two separate folders. But In the project dataset folder training and testing folders are presented. So, in this case, we just have to assign a variable and pass the folder path to it.
Four different transfer learning models are used in our project and the best model (Resnet50) is selected.

The image input size of xception model is 224, 224.

```
[ ] imageSize = [224, 224]
    #trainPath = "input/diabetic-retinopathy-level-detection/preprocessed dataset/preprocessed dataset/training"
    #testPath = "input/diabetic-retinopathy-level-detection/preprocessed dataset/preprocessed dataset/testing"
    trainPath = "/content/preprocessed dataset/preprocessed dataset/training"
    testPath = "/content/preprocessed dataset/preprocessed dataset/testing"
```

7.2 : Data pre-processing:

In this milestone we will be improving the image data that suppresses unwilling distortions or enhances some image features important for further processing, although perform some geometric transformations of images like rotation, scaling, translation, etc.

7.2.1 : Importing The Libraries

import the necessary libraries as shown in the image.

```
[ ] import tensorflow as tf
    from tensorflow import keras
    from tensorflow.keras.models import Sequential
    from tensorflow.keras.layers import Dense, Flatten, Input, Dropout
    from tensorflow.keras.models import Model,load_model
    from tensorflow.keras.preprocessing import image
    from tensorflow.keras.preprocessing.image import ImageDataGenerator, load_img
    from tensorflow.keras.applications.xception import Xception, preprocess_input
    from glob import glob
    import numpy as np
    import matplotlib.pyplot as plt
    import h5py
    import pickle
```

7.2.2 : Apply keras preprocessing functionality To Train Set And Test Set

Let us apply keras preprocessing functionality to the Train set and Test set by using the following code. For Training set using image_dataset_from_directory
function.
This function will return batches of images from the subdirectories
Arguments:

- directory: Directory where the data is located. If labels are "inferred", it should contain subdirectories, each containing images for a class. Otherwise, the directory structure is ignored. I have stored the directory in the variable ‘trainPath’ for training_data and ‘testPath’ for validation_data respectively.
- batch_size: Size of the batches of data which is 16.
- image_size: Size to resize images after they are read from disk.
- shuffle: To shuffle the data and set as True.
- Seed: used to set the random seed for shuffling the data during the creation of the image datasets
- Subset: used to specify whether the dataset being created is a subset of the entire dataset.
- Validation_split: used to specify the fraction of the dataset that should be reserved for validation. Set it 0.01 for our case.

```
training_data = keras.preprocessing.image_dataset_from_directory(
    trainPath,
    batch_size = 16,
    image_size =(224,224),

    shuffle = True,
    seed =123,
    subset ='training',
    validation_split=0.01
    )
validation_data =keras.preprocessing.image_dataset_from_directory(
    testPath,
    batch_size = 16,
    image_size =(224,224),

    shuffle = True,
    seed =123,
    validation_split =0.99,
    subset ='validation')
```

```
Found 3662 files belonging to 5 classes.
Using 3626 files for training.
Found 734 files belonging to 5 classes.
Using 726 files for validation.
```

7.3: Model Building:

Now it's time to build our Transfer Learning. We are using Resnet50 model to our project.

7.3.1: Pre-Trained TL Model As a Feature Extractor:

For one of the models, we will use it as a simple feature extractor by freezing all the five convolution blocks to make sure their weights don't get updated after each epoch as we train our own model.
Here, we have considered image_shape=(224,224,3).
Also, we have assigned include_top = False because we are using the convolution layer for features extraction and want to train a fully connected layer for our images classification(since it is not the part of Imagenet dataset)
pooling = 'avg'
classes=5
Flatten layer flattens the input. Does not affect the batch size.

```
[ ]  resnet_model = Sequential()

     pretrained_model= keras.applications.ResNet50(include_top=False,
                       input_shape=(224,224,3),
                       pooling='avg',classes=5,
                       weights='imagenet')
     for layer in pretrained_model.layers:
             layer.trainable=False

     resnet_model.add(pretrained_model)
     resnet_model.add(Flatten())
     resnet_model.add(Dense(128, activation='relu'))
     resnet_model.add(Dropout(0.5))
     resnet_model.add(Dense(5, activation='softmax'))

     Downloading data from https://storage.googleapis.com/tensorflow/keras-
     94765736/94765736 [==============================] - 0s 0us/step
```

7.3.2 : Configure The Learning Process:

The compilation is the final step in creating a model. Once the compilation is done, we can move on to the training phase. The loss function is used to find errors or deviations in the learning process. Keras requires a loss function during the model compilation process.
Optimization is an important process that optimizes the input weights by comparing the prediction and the loss function. Here we are using adam optimizer
Metrics are used to evaluate the performance of your model. It is similar to the loss function, but not used in the training process

```
[ ]  resnet_model.compile(loss='sparse_categorical_crossentropy', optimizer=keras.optimizers.SGD(lr=0.00009), metrics=['accuracy'])

     WARNING:absl:`lr` is deprecated in Keras optimizer, please use `learning_rate` or use the legacy optimizer, e.g.,tf.keras.optimizers.legacy.SGD.
```

7.3.3 : Train The Model:

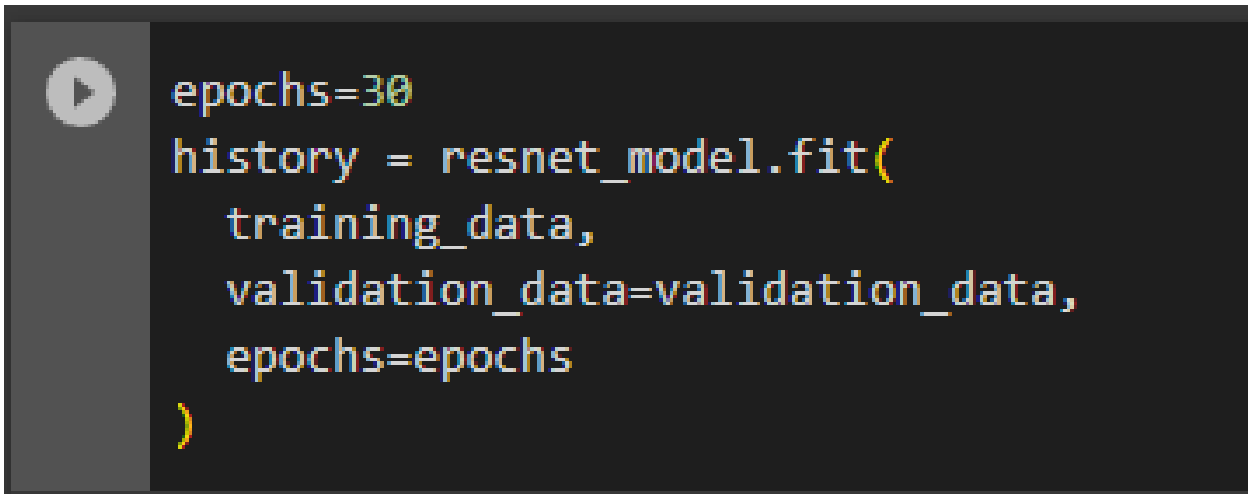
Now, let us train our model with our image dataset. The model is trained for 30 epochs and after every epoch, the current model state is saved if the model has the least loss encountered till that time. We can see that the training loss decreases in almost every epoch till 10 epochs and probably there is further scope to improve the model.
fit_generator functions used to train a deep learning neural network

Arguments:

- steps_per_epoch: it specifies the total number of steps taken from the generator as soon as one epoch is finished and the next epoch has started. We can calculate the value of steps_per_epoch as the total number of samples in your dataset divided by the batch size.

- Epochs: an integer and number of epochs we want to train our model for.

- validation_data can be either:
  - an inputs and targets list
  - a generator
  - inputs, targets, and sample_weights list which can be used to evaluate the loss and metrics for any model after any epoch has ended.
- validation_steps: only if the validation_data is a generator then only this argument can be used. It specifies the total number of steps taken from the generator before it is stopped at every epoch and its value is calculated as the total number of validation data points in your dataset divided by the validation batch size.



```
epochs=30
history = resnet_model.fit(
  training_data,
  validation_data=validation_data,
  epochs=epochs
)
```

```
Epoch 1/30
3/3 [==============================] - 32s 6s/step - loss: 9.6582 - accuracy: 0.4479
Epoch 2/30
3/3 [==============================] - 15s 5s/step - loss: 9.3711 - accuracy: 0.5417
Epoch 3/30
3/3 [==============================] - 15s 5s/step - loss: 7.4651 - accuracy: 0.5208
Epoch 4/30
3/3 [==============================] - 16s 5s/step - loss: 4.8766 - accuracy: 0.5938
Epoch 5/30
3/3 [==============================] - 15s 5s/step - loss: 6.3676 - accuracy: 0.6458
Epoch 6/30
3/3 [==============================] - 14s 5s/step - loss: 5.2558 - accuracy: 0.6667
Epoch 7/30
3/3 [==============================] - 16s 5s/step - loss: 5.4306 - accuracy: 0.6458
Epoch 8/30
3/3 [==============================] - 13s 4s/step - loss: 4.8280 - accuracy: 0.6562
Epoch 9/30
3/3 [==============================] - 14s 5s/step - loss: 3.9236 - accuracy: 0.6458
Epoch 10/30
3/3 [==============================] - 13s 4s/step - loss: 3.2804 - accuracy: 0.6562
Epoch 11/30
3/3 [==============================] - 15s 5s/step - loss: 2.1550 - accuracy: 0.6875
Epoch 12/30
3/3 [==============================] - 15s 5s/step - loss: 3.0436 - accuracy: 0.6979
Epoch 13/30
3/3 [==============================] - 14s 5s/step - loss: 3.4109 - accuracy: 0.7500
Epoch 14/30
3/3 [==============================] - 15s 5s/step - loss: 2.8810 - accuracy: 0.7396
Epoch 15/30
3/3 [==============================] - 15s 5s/step - loss: 3.4979 - accuracy: 0.6667
Epoch 16/30
3/3 [==============================] - 14s 5s/step - loss: 3.1029 - accuracy: 0.6562
Epoch 17/30
3/3 [==============================] - 14s 5s/step - loss: 2.8477 - accuracy: 0.6979
Epoch 18/30
3/3 [==============================] - 14s 4s/step - loss: 2.6290 - accuracy: 0.6979
Epoch 19/30
3/3 [==============================] - 16s 5s/step - loss: 4.5827 - accuracy: 0.5938
Epoch 20/30
3/3 [==============================] - 13s 4s/step - loss: 1.7713 - accuracy: 0.7604
Epoch 21/30
3/3 [==============================] - 14s 5s/step - loss: 4.1266 - accuracy: 0.6042
Epoch 22/30
3/3 [==============================] - 15s 5s/step - loss: 2.2045 - accuracy: 0.7188
Epoch 23/30
3/3 [==============================] - 15s 5s/step - loss: 2.7497 - accuracy: 0.7500
Epoch 24/30
3/3 [==============================] - 16s 5s/step - loss: 3.3502 - accuracy: 0.7083
Epoch 25/30
3/3 [==============================] - 15s 5s/step - loss: 3.1592 - accuracy: 0.7188
Epoch 26/30
3/3 [==============================] - 14s 5s/step - loss: 3.2060 - accuracy: 0.6354
Epoch 27/30
3/3 [==============================] - 16s 5s/step - loss: 3.4886 - accuracy: 0.6250
Epoch 28/30
3/3 [==============================] - 15s 5s/step - loss: 3.0558 - accuracy: 0.6979
Epoch 29/30
3/3 [==============================] - 14s 5s/step - loss: 4.7360 - accuracy: 0.6250
Epoch 30/30
3/3 [==============================] - 14s 5s/step - loss: 2.0049 - accuracy: 0.7708
```

7.3.4 : Saving the model:

The model is saved with .h5 extension as follows
An H5 file is a data file saved in the Hierarchical Data Format (HDF). It contains multidimensional arrays of scientific data.

```
resnet_model.save('inception-diabetic.h5')
```

```
/usr/local/lib/python3.10/dist-packages/keras/
  saving_api.save_model(
```

7.3.5 : Testing the model:

Evaluation is a process during the development of the model to check whether the model is the best fit for the given problem and corresponding data.
Load the saved model using load_model
Taking an image as input and checking the results
By using the model we are predicting the output for the given input image
The predicted class index name will be printed here.

Testing the model

```
[13] model = load_model('inception-diabetic.h5')
```

Loading a image from class 0

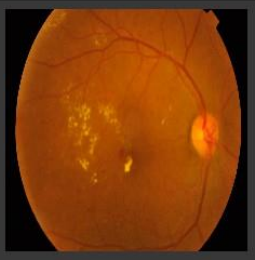
```
i = image.load_img('/content/preprocessed dataset/preprocessed dataset/testing/0/d160ebef4117.png', target_size=(224,224))
i
```

```
[31] img_array = image.img_to_array(i)
     img_array = np.expand_dims(img_array, axis=0)
     img_array = img_array / 255.0
     predictions = model.predict(img_array)
     predicted_class = np.argmax(predictions)
     class_labels = [
                     "No apparent Diabetic retinopathy",
                     "Mild nonproliferative Diabetic retinopathy (Mild NPDR)",
                     "Moderate nonproliferative Diabetic retinopathy (Moderate NPDR)",
                     "Severe nonproliferative Diabetic retinopathy (Severe NPDR)",
                     "Proliferative Diabetic retinopathy (PDR)",
                 ]
     predicted_label = class_labels[predicted_class]
     print("Predicted class:", predicted_label)

     1/1 [==============================] - 0s 27ms/step
     Predicted class: No apparent Diabetic retinopathy
```

Loading a image from class 1

```
[32] i = image.load_img('/content/preprocessed dataset/preprocessed dataset/testing/1/d4f32b9c07df.png', target_size=(224,224))
     i
```



```
img_array = image.img_to_array(i)
img_array = np.expand_dims(img_array, axis=0)
img_array = img_array / 255.0
predictions = model.predict(img_array)
predicted_class = np.argmax(predictions)
class_labels = [
                "No apparent Diabetic retinopathy",
                "Mild nonproliferative Diabetic retinopathy (Mild NPDR)",
                "Moderate nonproliferative Diabetic retinopathy (Moderate NPDR)",
                "Severe nonproliferative Diabetic retinopathy (Severe NPDR)",
                "Proliferative Diabetic retinopathy (PDR)",
            ]
predicted_label = class_labels[predicted_class]
print("Predicted class:", predicted_label)
```

```
1/1 [==============================] - 0s 27ms/step
Predicted class: Moderate nonproliferative Diabetic retinopathy (Moderate NPDR)
```

Loading a image from class 2

```
[34] i = image.load_img('/content/preprocessed dataset/preprocessed dataset/testing/2/cac40227d3b2.png', target_size=(224,224))
     i
```

```
img_array = image.img_to_array(i)
img_array = np.expand_dims(img_array, axis=0)
img_array = img_array / 255.0
predictions = model.predict(img_array)
predicted_class = np.argmax(predictions)
class_labels = [
                "No apparent Diabetic retinopathy",
                "Mild nonproliferative Diabetic retinopathy (Mild NPDR)",
                "Moderate nonproliferative Diabetic retinopathy (Moderate NPDR)",
                "Severe nonproliferative Diabetic retinopathy (Severe NPDR)",
                "Proliferative Diabetic retinopathy (PDR)",
            ]
predicted_label = class_labels[predicted_class]
print("Predicted class:", predicted_label)
```

```
1/1 [==============================] - 0s 27ms/step
Predicted class: Moderate nonproliferative Diabetic retinopathy (Moderate NPDR)
```

Loading a image from class 3

```
i = image.load_img('/content/preprocessed dataset/preprocessed dataset/testing/3/c31651ea04c6.png', target_size=(224,224))
i
```

```
img_array = image.img_to_array(i)
img_array = np.expand_dims(img_array, axis=0)
img_array = img_array / 255.0
predictions = model.predict(img_array)
predicted_class = np.argmax(predictions)
class_labels = [
                "No apparent Diabetic retinopathy",
                "Mild nonproliferative Diabetic retinopathy (Mild NPDR)",
                "Moderate nonproliferative Diabetic retinopathy (Moderate NPDR)",
                "Severe nonproliferative Diabetic retinopathy (Severe NPDR)",
                "Proliferative Diabetic retinopathy (PDR)",
            ]
predicted_label = class_labels[predicted_class]
print("Predicted class:", predicted_label)
```

```
1/1 [==============================] - 0s 43ms/step
Predicted class: Severe nonproliferative Diabetic retinopathy (Severe NPDR)
```

Loading a image from class 4

```
i = image.load_img('/content/preprocessed dataset/preprocessed dataset/testing/4/d85ea1220a03.png', target_size=(224,224))
i
```

```
img_array = image.img_to_array(i)
img_array = np.expand_dims(img_array, axis=0)
img_array = img_array / 255.0
predictions = model.predict(img_array)
predicted_class = np.argmax(predictions)
class_labels = [
                "No apparent Diabetic retinopathy",
                "Mild nonproliferative Diabetic retinopathy (Mild NPDR)",
                "Moderate nonproliferative Diabetic retinopathy (Moderate NPDR)",
                "Severe nonproliferative Diabetic retinopathy (Severe NPDR)",
                "Proliferative Diabetic retinopathy (PDR)",
            ]
predicted_label = class_labels[predicted_class]
print("Predicted class:", predicted_label)
```

```
1/1 [==============================] - 0s 28ms/step
Predicted class: Proliferative Diabetic retinopathy (PDR)
```

Chapter 8

Testing & Validation

8.1: Manual Testing

Test Case ID	Test Scenario	Input	Expected Result	Actual Result	Status
TC01	Upload valid retinal image	JPEG image	Image accepted	Image accepted	Pass
TC02	Upload PNG image	PNG image	Image accepted	Image accepted	Pass
TC03	Upload invalid file	PDF file	Error message	Error displayed	Pass
TC04	Submit without image	No file	Validation error	Validation error	Pass
TC05	Prediction generation	Valid image	DR class displayed	Class displayed	Pass

8.2: Unit Testing

Image Upload Module

Test	Expected
Valid image upload	Success
Unsupported format	Unsupported format

Preprocessing Module

Test	Expected
Resize image	224×224 output
Normalize pixels	Values between 0 and 1

Prediction Module

Test	Expected
Model receives image	Returns class probabilities
Softmax output	5 class scores

8.3: Integration Testing

Integration Test	Expected Result
Upload → Preprocessing	Image processed correctly
Preprocessing → ResNet50	Model receives valid tensor
Model → Recommendation Engine	Recommendation generated
Backend → Frontend	Results displayed

8.4: Performance Testing

Parameter	Result
Deep Learning Model	ResNet50 Transfer Learning
Dataset Source	Kaggle Diabetic Retinopathy Dataset
Number of Classes	5
Training Images	3626
Validation Images	726
Input Image Size	224 × 224 Pixels
Batch Size	16
Number of Epochs	10
Training Accuracy	78.68%
Validation Accuracy	81.68%
Classification Type	Multi-Class Classification
Output Categories	5 Severity Levels
Framework	TensorFlow / Keras
Backend Framework	Flask
Prediction Mode	Real-Time Inference
Image Format Supported	JPG, JPEG, PNG
Model File Format	H5
Deployment Platform	Local Flask Web Application

8.5: Functional Testing

Test Case	Expected Result	Status
Upload Valid Fundus Image	Image uploaded successfully	Pass
Predict Disease Severity	Correct severity class displayed	Pass
Display Recommendation	Recommendation displayed	Pass
Upload PNG Image	Successfully processed	Pass
Upload JPG Image	Successfully processed	Pass
Multiple Predictions	Predictions generated successfully	Pass
Load Trained Model	Model loaded successfully	Pass
Web Application Launch	Application starts successfully	Pass

8.6: User Acceptance Testing

Test Case	Result
Upload JPG Image	Pass
Upload PNG Image	Pass
Invalid File Upload	Pass
Prediction Generation	Pass
Recommendation Display	Pass
Browser Compatibility (Chrome)	Pass
Browser Compatibility (Edge)	Pass

8.7: Model Accuracy Summary

Metric	Value
Training Accuracy	78.68%
Validation Accuracy	81.68%
Number of Classes	5
Epochs	10
Transfer Learning Architecture	ResNet50
Pretrained Weights	ImageNet

Chapter 9

Results & Screenshots

This chapter presents the key functional screenshots and results of the DRetina Check system, demonstrating the successful implementation of diabetic retinopathy detection using deep learning. The screenshots provide visual evidence of the complete workflow, including retinal fundus image upload, image preprocessing, disease classification, and result generation through the web application.

The results validate the effectiveness of the proposed system in performing automated diabetic retinopathy screening. The application successfully integrates a trained deep learning model with a user-friendly web interface, enabling real-time prediction of disease severity from retinal fundus images. The screenshots highlight the seamless interaction between the user interface, Flask backend, image preprocessing pipeline, and deep learning model.

By displaying actual prediction outputs and system interactions, the screenshots demonstrate how the application fulfills its objective of assisting in the early detection of diabetic retinopathy and supporting healthcare professionals in preliminary screening.

User Interface
The user interface is designed to provide a simple and intuitive experience for users. Through a web-based platform, users can upload retinal fundus images for analysis. The interface ensures easy navigation and quick access to prediction results, making the system suitable for both healthcare professionals and non-technical users.
The homepage provides project information and image upload functionality. Once an image is uploaded, the system processes the image and displays the predicted diabetic retinopathy severity level along with general recommendations.

Prediction Interface
The prediction interface serves as the core functionality of the application. After receiving a retinal image, the Flask backend performs image preprocessing, including resizing the image to 224×224 pixels and normalizing pixel values. The processed image is then passed to the trained deep learning model for classification.

The system classifies retinal images into one of the following five categories:

- No Apparent Diabetic Retinopathy
- Mild Non-Proliferative Diabetic Retinopathy (Mild NPDR)
- Moderate Non-Proliferative Diabetic Retinopathy (Moderate NPDR)
- Severe Non-Proliferative Diabetic Retinopathy (Severe NPDR)
- Proliferative Diabetic Retinopathy (PDR)
  The prediction result is displayed instantly along with appropriate general recommendations related to the identified severity level.

Deep Learning Model Performance

The system utilizes Transfer Learning with a pretrained ResNet50 architecture for feature extraction and classification. The model was trained on a diabetic retinopathy dataset containing retinal fundus images distributed across five disease severity classes.

The final model achieved:

- Training Accuracy: 78.68%
- Validation Accuracy: 81.68%
- Number of Classes: 5
- Training Images: 3626
- Validation Images: 726
- Number of Epochs: 10

These results demonstrate the capability of the model to effectively learn discriminative retinal features and perform multi-class diabetic retinopathy classification.

Challenges Overcome

Several challenges were addressed during the development of the system. One major challenge was handling the variability in retinal image quality, illumination, and contrast. Image preprocessing techniques were applied to ensure consistency across different samples.

Another challenge involved training an accurate deep learning model using a limited medical imaging dataset. This was addressed through Transfer Learning using a pretrained ResNet50 model, which improved feature extraction and reduced training time.

Integrating the trained model with the Flask web framework and ensuring smooth communication between the frontend and backend components also required careful implementation. Extensive testing was performed to ensure reliable image uploads, prediction generation, and result presentation.

Overall, the successful deployment and testing of DRetina Check demonstrate its potential as an automated screening tool for the early detection of diabetic retinopathy.

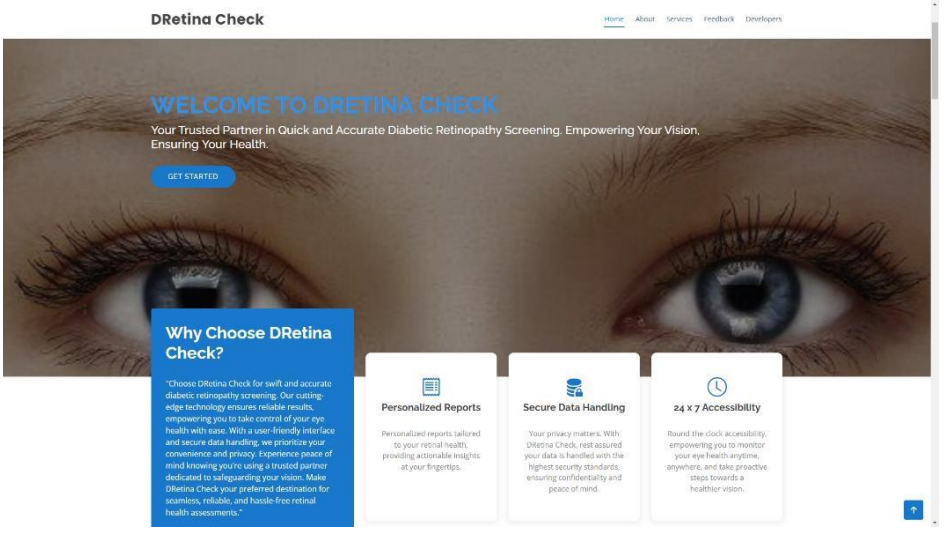


Fig 9.1: The User Interface

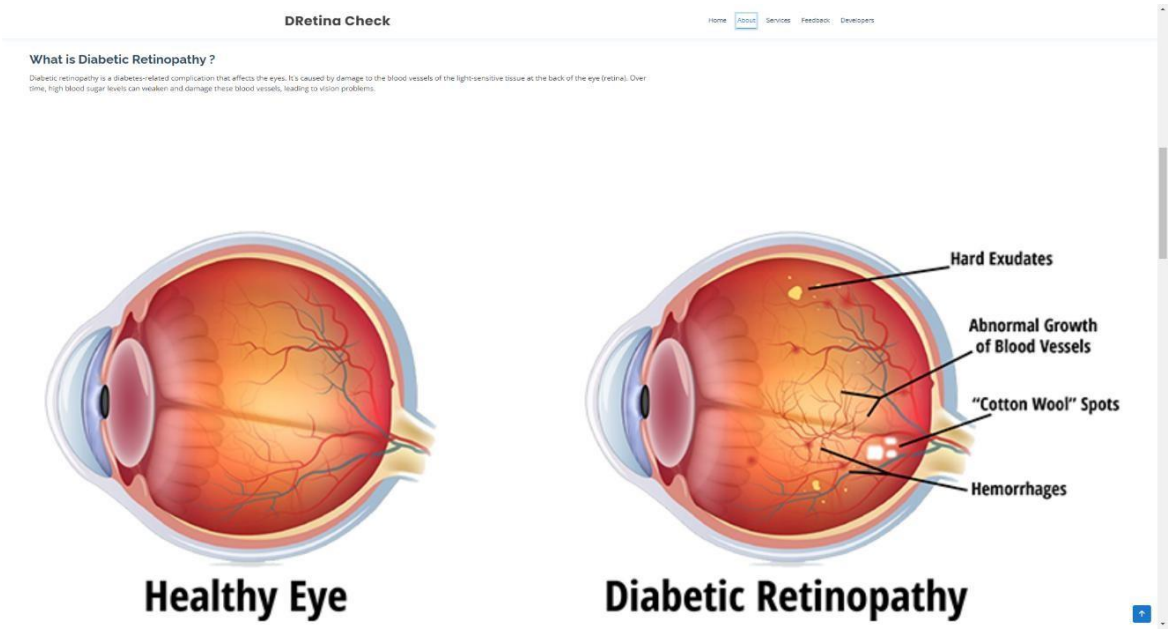


Fig 9.2: What is Retinopathy

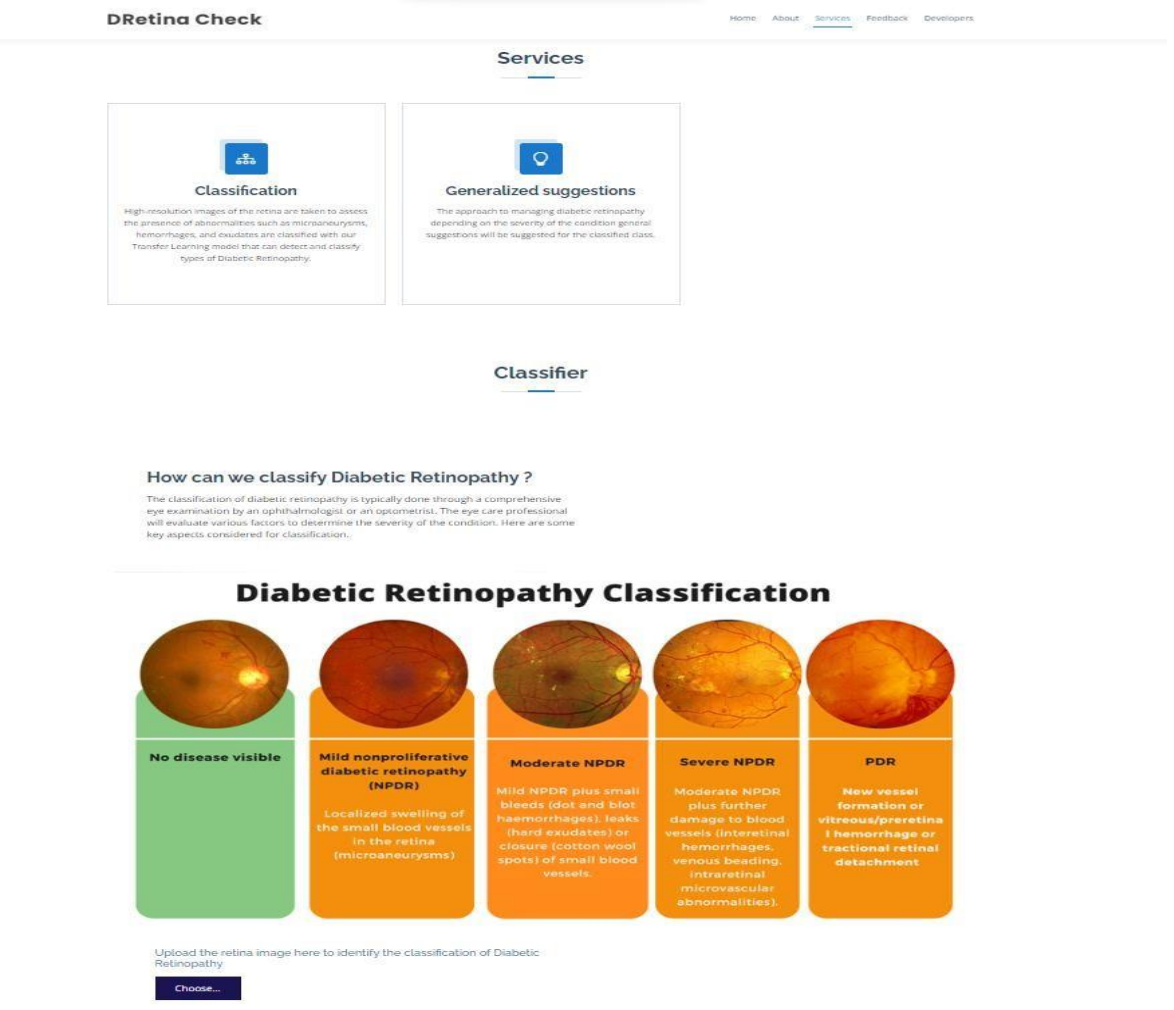


Fig 9.3: Services and identification of the disease

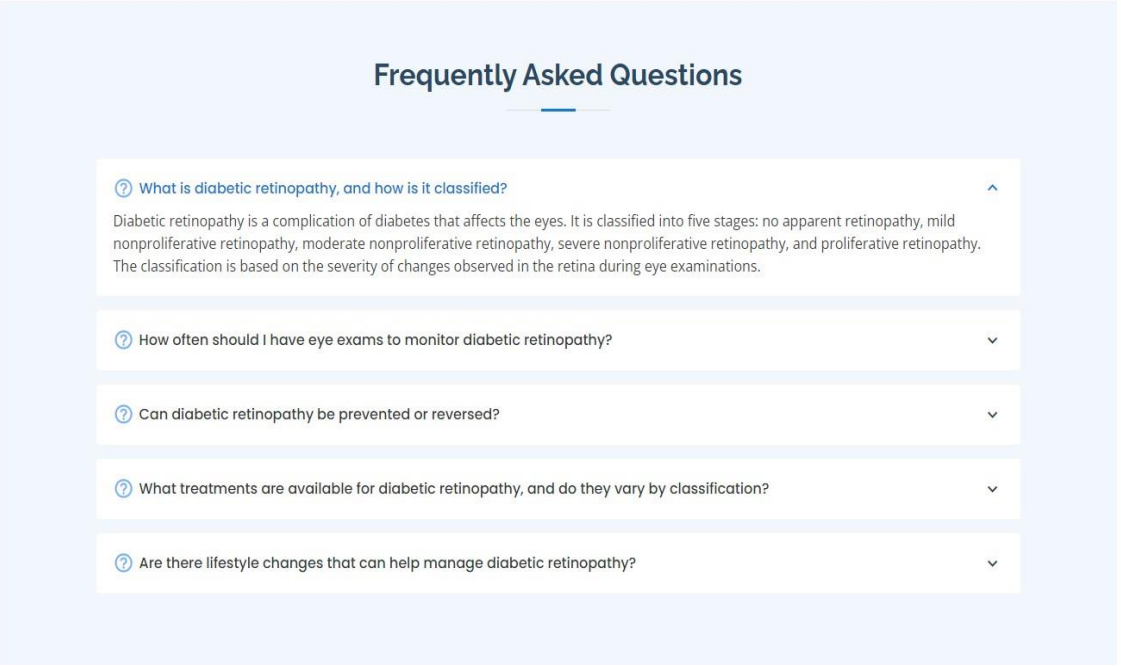


Fig 9.4: Frequently Asked Questions

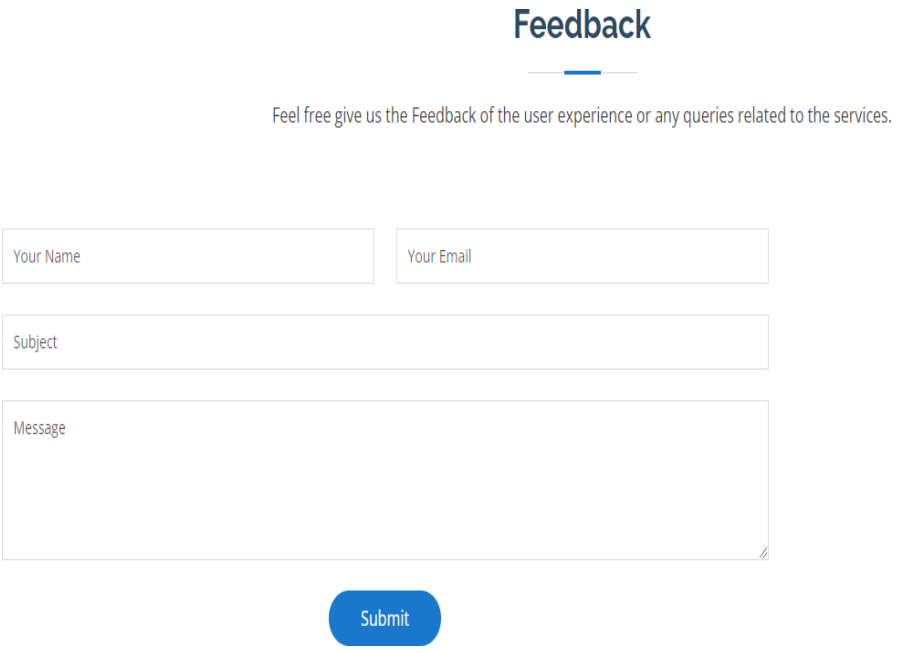


Fig 9.5: Feedback

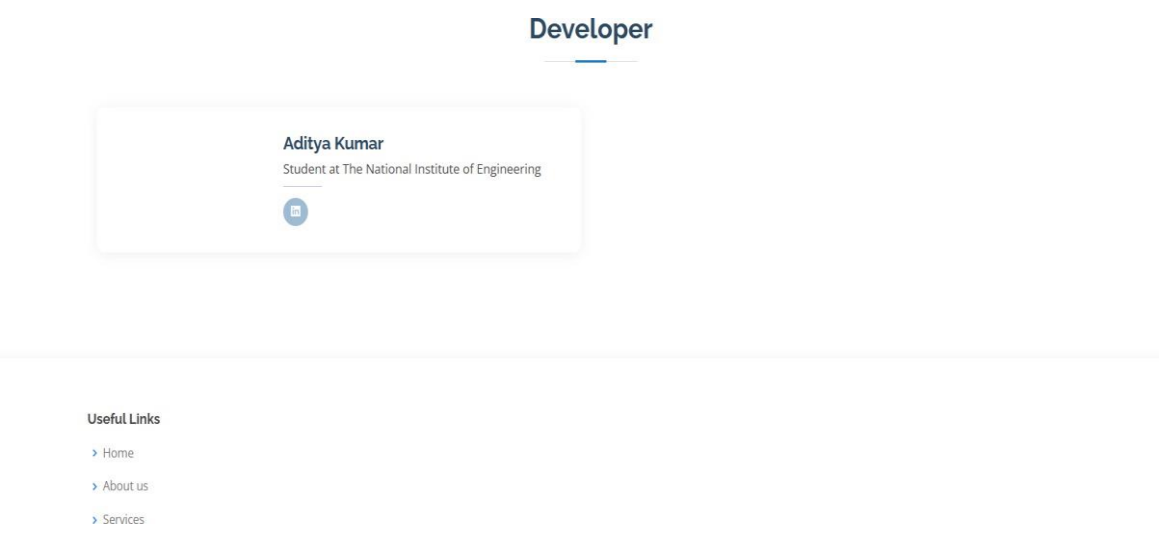


Fig 9.6: Developer & Footer Section

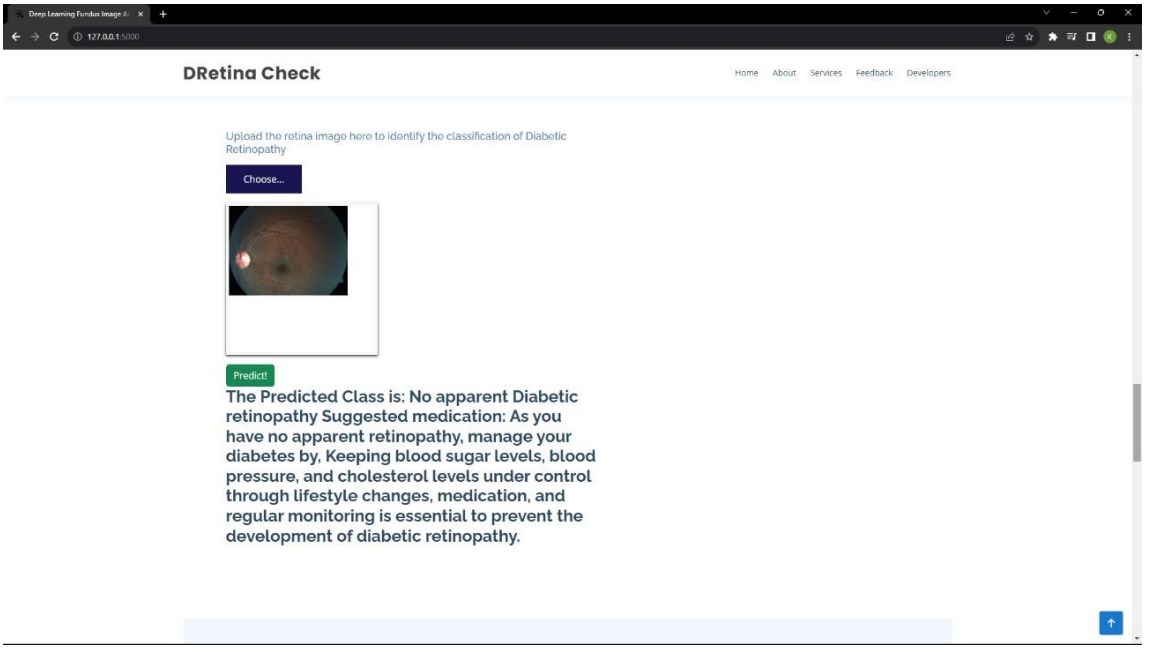


Fig 9.7: No apparent Diabetic retinopathy

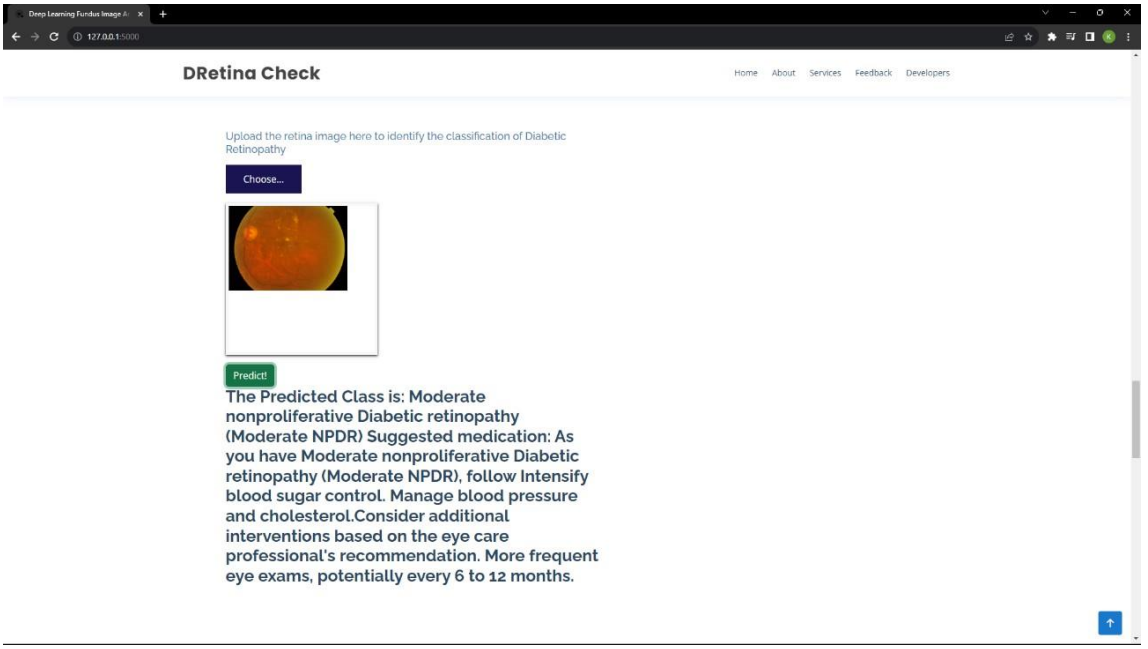


Fig 9.8: Moderate nonproliferative Diabetic retinopathy

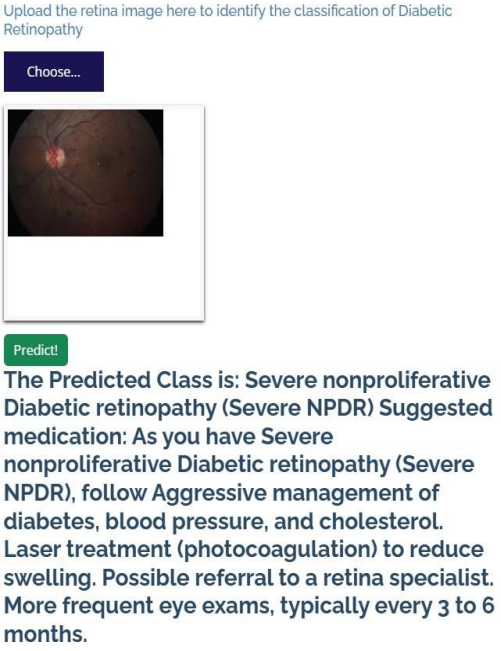


Fig 9.9: Severe nonproliferative Diabetic retinopathy

CONCLUSION

DRetina Check demonstrates the practical and effective application of deep learning and transfer learning techniques to the domain of medical image analysis. The project successfully delivers a web-based application that automates the detection and five-class severity classification of Diabetic Retinopathy from retinal fundus images, addressing the key limitations of existing manual and automated screening approaches.

The system integrates a ResNet50-based Convolutional Neural Network pre-trained on the ImageNet dataset with a modular Flask backend and a responsive web frontend. The preprocessing pipeline ensures that uploaded retinal images are correctly resized, converted, and normalised before inference, producing reliable and consistent classification results. The recommendation module provides severity-specific general guidance that encourages users to seek appropriate medical consultation based on the predicted outcome.

The five-class classification framework adopted in DRetina Check aligns with the internationally recognised Diabetic Retinopathy grading scale and provides significantly more clinically meaningful output than binary normal or abnormal classification systems. By leveraging transfer learning, the system achieves effective feature extraction from retinal images without requiring an excessively large annotated training dataset, making it a practical solution for real-world deployment in resource-constrained healthcare settings.

The modular architecture of the system ensures that individual components can be independently updated, extended, or replaced without disrupting the overall workflow. This design positions DRetina Check as a strong foundation for future enhancements including Grad-CAM explainability visualisation, cloud deployment, patient record management, mobile application support, and extension to additional retinal diseases such as glaucoma and age-related macular degeneration.

DRetina Check is not intended to replace qualified ophthalmologists or serve as a definitive diagnostic tool. Rather, it functions as an accessible, fast, and cost-effective preliminary screening support system that can assist healthcare professionals, reduce diagnostic workload, and encourage diabetic patients to seek timely medical evaluation. In regions with limited access to specialist eye care, such a system has significant potential to improve early detection rates and reduce the burden of preventable vision loss caused by Diabetic Retinopathy.

Overall, this project demonstrates that open-source deep learning technologies, when thoughtfully applied to well-defined clinical problems, can produce practical healthcare.

Future Enhancements

Although DRetina Check successfully performs automated diabetic retinopathy classification using deep learning, several enhancements can be implemented in the future to improve its accuracy, usability, and real-world applicability.

- Improved Model Accuracy
  The current system can be enhanced by training on larger and more diverse retinal image datasets. Advanced deep learning architectures such as EfficientNet, Vision Transformers (ViT), or ensemble models can be explored to improve classification performance and robustness.

- Explainable AI Integration
  Explainable AI techniques such as Grad-CAM and heatmap visualization can be incorporated to highlight the retinal regions responsible for the model's predictions. This would increase transparency and help ophthalmologists better understand the decision-making process of the model.

- Mobile Application Development
  A dedicated Android and iOS application can be developed to allow healthcare professionals and patients to perform retinal image screening directly from mobile devices, increasing accessibility and convenience.

- Cloud-Based Deployment
  The application can be deployed on cloud platforms to enable remote access, scalability, and real-time predictions from multiple locations without requiring local installation.

- Electronic Health Record Integration
  Integration with Electronic Health Record (EHR) systems can allow automatic storage and retrieval of patient data, improving clinical workflow and patient management.

- Multi-Disease Retinal Analysis
  The system can be extended to detect additional retinal diseases such as Glaucoma, Age-Related Macular Degeneration (AMD), Cataracts, and Hypertensive Retinopathy, making it a comprehensive ophthalmic screening platform.

- Patient Monitoring Dashboard
  A dashboard can be developed to track disease progression over time by maintaining historical retinal scans and generating comparative analysis reports for doctors and patients.

- Automated Report Generation
  The application can generate detailed PDF reports containing prediction results, confidence scores, retinal image analysis, and clinical recommendations for easier documentation and record keeping.

- Telemedicine Integration
  Future versions can support telemedicine services, enabling remote consultations between patients and ophthalmologists based on the generated screening results.

- Clinical Validation and Deployment
  Extensive clinical validation using larger real-world datasets and collaboration with healthcare institutions can further improve reliability and support deployment in hospitals and diagnostic centers.

These enhancements would significantly increase the practical utility of DRetina Check and strengthen its role as an intelligent decision-support system for early detection and management of diabetic retinopathy.

References

[1] Kaggle, "Diabetic Retinopathy Detection Dataset." Available: https://www.kaggle.com/datasets/arbethi/diabetic-retinopathy-level-detection

[2] K. He, X. Zhang, S. Ren, and J. Sun, "Deep Residual Learning for Image Recognition," Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR), pp. 770–778, 2016.

[3] F. Chollet et al., "Keras: Deep Learning for Humans." Available: https://keras.io

[4] TensorFlow Developers, "TensorFlow: An End-to-End Open Source Machine Learning Platform." Available: https://www.tensorflow.org

[5] M. D. Abràmoff, M. K. Garvin, and M. Sonka, "Retinal Imaging and Image Analysis," IEEE Reviews in Biomedical Engineering, vol. 3, pp. 169–208, 2010.

[6] V. Gulshan et al., "Development and Validation of a Deep Learning Algorithm for Detection of Diabetic Retinopathy in Retinal Fundus Photographs," Journal of the American Medical Association (JAMA), vol. 316, no. 22, pp. 2402–2410, 2016.

[7] T. Y. Wong and N. M. Bressler, "Artificial Intelligence with Deep Learning Technology Looks into Diabetic Retinopathy Screening," JAMA, vol. 316, no. 22, pp. 2366–2367, 2016.

[8] Flask Documentation, "Flask Web Development Framework." Available: https://flask.palletsprojects.com

[9] Python Software Foundation, "Python Programming Language." Available: https://www.python.org

[10] A. Krizhevsky, I. Sutskever, and G. E. Hinton, "ImageNet Classification with Deep Convolutional Neural Networks," Advances in Neural Information Processing Systems (NIPS), vol. 25, pp. 1097–1105, 2012.

[11] J. Brownlee, "A Gentle Introduction to Transfer Learning for Deep Learning." Available: https://machinelearningmastery.com/transfer-learning-for-deep-learning

[12] Pillow Documentation, "Python Imaging Library (PIL)." Available: https://pillow.readthedocs.io

[13] NumPy Developers, "NumPy: Fundamental Package for Scientific Computing with Python." Available: https://numpy.org

[14] Bootstrap Documentation, "Bootstrap Front-End Framework." Available: https://getbootstrap.com

[15] World Health Organization (WHO), "Blindness and Vision Impairment." Available: https://www.who.int